HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA VIỄN THÔNG 1

****

BÁO CÁO ĐỀ TÀI

KHOA HỌC CÔNG NGHỆ SINH VIÊN 2025

ĐỀ TÀI:

NGHIÊN CỨU CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ (GNN)

Mã số: 31 – SV – 2025 – VT1

Nhóm thực hiện:	**Hà Thanh Tùng**	**D23CQKD02**
	Phạm Minh Kha	**D25CQCN04**

Giảng viên hướng dẫn: ThS. Bùi Trường Sơn

Hà Nội 10/2025

NHẬN XÉT, ĐÁNH GIÁ
(CỦA GIÁO VIÊN HƯỚNG DẪN)

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Hà Nội, ngày tháng 11 năm 2025

CÁN BỘ - GIẢNG VIÊN HƯỚNG DẪN

(ký và ghi rõ họ tên)

Bùi Trường Sơn

NHẬN XÉT, ĐÁNH GIÁ, CHO ĐIỂM
(CỦA GIÁO VIÊN PHẢN BIỆN)

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Điểm:……(Bằng chữ:……..)

Hà Nội, ngày tháng 11 năm 2025

CÁN BỘ - GIẢNG VIÊN PHẢN BIỆN

(ký và ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN 3

LỜI MỞ ĐẦU 4

MỤC LỤC HÌNH ẢNH 6

MỤC LỤC VIẾT TẮT 7

CHƯƠNG 1: TỔNG QUAN VỀ CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ 8

1.1. Giới thiệu về mạng nơ-ron đồ thị 8

1.1.1. Bối cảnh ra đời và sự cần thiết của GNN 8

1.1.2. Khái niệm cơ bản về đồ thị trong học sâu 9

1.1.3. Ứng dụng của mạng nơ-ron đồ thị trong thực tế 11

1.2. Lịch sử phát triển và các hướng nghiên cứu hiện nay 14

1.2.1. Lịch sử phát triển của GNN 14

1.2.2. Các hướng phát triển hiện nay 15

1.2.3. Thách thức trong nghiên cứu và ứng dụng 19

1.3. Kết luận 20

CHƯƠNG 2: CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ 22

2.1. Cơ sở lý thuyết về mạng nơ-ron đồ thị 22

2.1.1. Biểu diễn toán học của đồ thị 22

2.1.2. Cơ chế lan truyền thông tin trên đồ thị 23

2.1.3. Kiến trúc tổng quát của mô hình GNN 25

2.2. Các mô hình mạng nơ-ron đồ thị phổ biến 25

2.2.1. Graph Convolutional Network (GCN) 25

2.2.2. Graph Attention Network 29

2.2.3. GraphSAGE và các biến thể 32

2.3. Kết luận 35

CHƯƠNG 3: MÔ PHỎNG VÀ THỬ NGHIỆM CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ 36

3.1. Giới thiệu phần mềm và công cụ hỗ trợ quá trình mô phỏng 36

3.1.1. Visual Studio Code 36

3.1.2. Jupyter Notebook 37

3.1.3. Thư viện hỗ trợ 38

3.2. Cài đặt và thiết lập mô phỏng 39

3.2.1. Chuẩn bị dữ liệu 39

3.2.2. Huấn luyện mô hình và kết quả 40
3.3. Kết luận 44
TỔNG KẾT **45**
TÀI LIỆU THAM KHẢO **46**

LỜI CẢM ƠN

Trong suốt thời gian từ khi bắt đầu học tập ở Học viện đến nay, chúng em đã nhận được rất nhiều sự quan tâm, giúp đỡ của quý thầy cô trong Học viện nói chung và đặt biệt là thầy cô khoa Viễn Thông I nói riêng đã cùng với tri thức và sự tâm huyết của mình để truyền đạt vốn kiến thức quý báu cho chúng em. Những kiến thức tiếp thu trong quá trình học tập không chỉ là nền tảng cho quá trình nghiên cứu khoa học mà còn là hành trang quý báu để chúng em bước vào cuộc sống một cách vững chắc và tự tin hơn.

Chúng em xin chân thành cảm ơn **ThS. Bùi Trường Sơn**. Thầy đã tận tâm hướng dẫn chúng em trong suốt quá trình làm nghiên cứu của mình. Nhờ vậy mà chúng em đã hoàn thành bài nghiên cứu một cách tốt nhất. Một lần nữa em xin chân thành cảm ơn thầy.

Mặc dù đã có nhiều cố gắng để thực hiện đề tài một cách hoàn chỉnh nhất. Song do buổi đầu mới làm quen với công tác nghiên cứu khoa học, kiến thức cũng như kinh nghiệm của chúng em còn nhiều hạn chế và bỡ ngỡ. Do vậy chắc chắn sẽ không thể tránh khỏi những thiếu sót mà bản thân chưa thấy được. Chúng em rất mong được sự góp ý từ quý thầy cô để kiến thức của chúng em trong lĩnh vực này được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện đề tài

LỜI MỞ ĐẦU

Trong bối cảnh thế giới công nghệ số phát triển mạnh mẽ, dữ liệu đang trở thành yếu tố cốt lõi thúc đẩy sự tiến bộ của nhiều lĩnh vực như trí tuệ nhân tạo, phân tích dữ liệu, và tự động hóa. Sự gia tăng nhanh chóng của các hệ thống thông tin, mạng xã hội, cảm biến IoT hay các hệ thống khuyến nghị đã tạo ra lượng dữ liệu khổng lồ, không chỉ đa dạng về nội dung mà còn phức tạp về cấu trúc. Trong khi các mô hình học sâu truyền thống như mạng nơ-ron tích chập hay mạng nơ-ron hồi quy chủ yếu hoạt động hiệu quả trên dữ liệu có cấu trúc dạng lưới, thì một phần lớn dữ liệu thực tế lại mang đặc trưng phi cấu trúc hoặc phi Euclid, được biểu diễn tự nhiên dưới dạng đồ thị.

Chính từ nhu cầu đó, mạng nơ-ron đồ thị ra đời, mở ra một hướng nghiên cứu mới trong lĩnh vực học sâu. Mạng nơ-ron đồ thị cho phép mô hình hóa và học biểu diễn trên dữ liệu đồ thị, trong đó thông tin của mỗi nút không chỉ phụ thuộc vào đặc trưng riêng mà còn chịu ảnh hưởng từ các nút lân cận thông qua các cạnh liên kết. Cơ chế lan truyền thông tin này giúp mạng nơ-ron đồ thị nắm bắt được mối quan hệ và cấu trúc toàn cục của đồ thị, mang lại hiệu quả vượt trội trong các bài toán như phân loại nút, dự đoán liên kết, phân cụm đồ thị, hệ thống khuyến nghị, và phân tích mạng xã hội.

Tuy nhiên, cùng với những tiềm năng to lớn, mạng nơ-ron đồ thị cũng đối mặt với nhiều thách thức. Việc mở rộng mô hình lên các đồ thị lớn đòi hỏi khả năng tính toán cao; vấn đề over-smoothing hay over-squashing có thể làm giảm khả năng biểu diễn của mô hình; đồng thời, việc thiết kế các kiến trúc mạng nơ-ron đồ thị hiệu quả cho từng loại dữ liệu và bài toán vẫn là chủ đề được quan tâm trong cộng đồng nghiên cứu.

Đề tài "Nghiên cứu về mạng nơ-ron đồ thị (GNN)" ra đời với mục tiêu tìm hiểu sâu về cơ sở lý thuyết, nguyên lý hoạt động và các biến thể của mạng nơ-ron đồ thị, từ đó tiến hành mô phỏng và đánh giá hiệu năng của một số mô hình tiêu biểu. Tài liệu sẽ cung cấp một cái nhìn tổng quan, trình bày các cơ sở toán học và lý thuyết nền tảng, đồng thời phân tích kết quả thực nghiệm để làm rõ khả năng ứng dụng của mạng nơ-ron đồ thị trong xử lý dữ liệu phức tạp.

Về cơ bản, đề tài này được tổ chức theo ba chương chính:

Chương I: Tổng quan về công nghệ mạng nơ-ron đồ thị

Chương II: Công nghệ mạng nơ-ron đồ thị

Chương III: Xây dựng mô phỏng mạng nơ-ron đồ thị

Thông qua nghiên cứu này, nhóm chúng em mong muốn mang đến một cái nhìn toàn diện về mạng nơ-ron đồ thị (GNN) - một lĩnh vực đang phát triển nhanh chóng trong nghiên cứu trí tuệ nhân tạo. Đề tài hướng tới việc giúp người đọc hiểu rõ cơ chế hoạt động, ưu điểm và tiềm năng ứng dụng của mạng nơ-ron đồ thị trong xu hướng công nghiệp hiện đại ngày nay.

MỤC LỤC HÌNH ẢNH

Hình 1 : Tổng quan về công nghệ mạng nơ ron đồ thị 8
Hình 2 : Các thành phần cơ bản của đồ thị 10
Hình 3 : Các ứng dụng của mạng nơ-ron đồ thị 12
Hình 4 : Lịch sử phát tiển của GNN 14
Hình 5 : Mô hình GCN 26
Hình 6 : Mô hình GAT 30
Hình 7 : Quy trình hoạt động của mô hình GraphSAGE 33
Hình 8 : Visual Studio Code 36
Hình 9 : Jupyter Notebook 37
Hình 10 : Lấy dataset về để mô phỏng 40
Hình 11 : Định nghĩa lớp mô hình GCN 41
Hình 12 : Cấu hình tối ưu trong quá trình huấn luyện 41
Hình 13 : Quy trình huấn luyện mô hình GCN qua các epoch 42
Hình 14 : Biểu đồ giá trị hàm mất mát 42
Hình 15 : Biểu đồ độ chính xác của mô hình 43

MỤC LỤC VIẾT TẮT

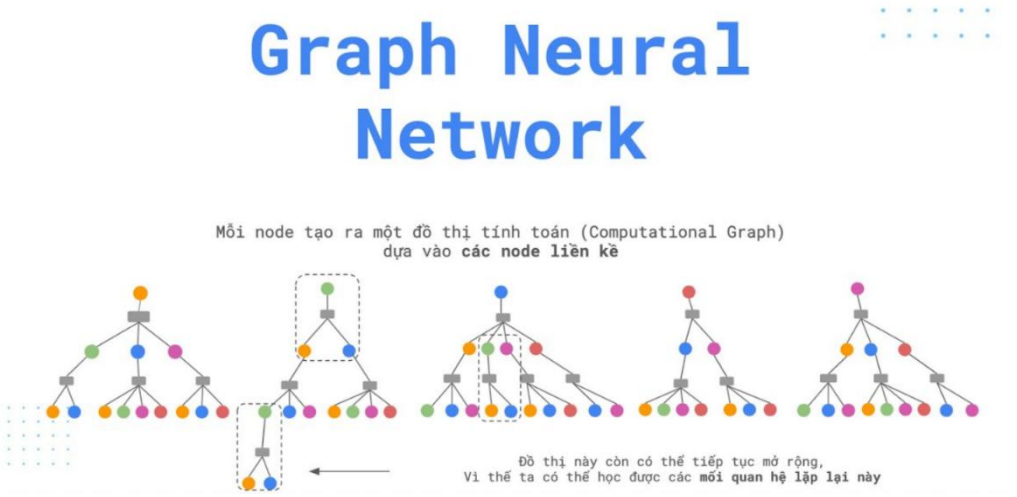
Từ viết tắt	Thuật ngữ Tiếng Anh	Thuật ngữ Tiếng Việt
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
GCN	Graph Convolutional Network	Mạng nơ-ron tích chập trên đồ thị
GAT	Graph Attention Network	Mạng nơ-ron chú ý trên đồ thị
GraphSAGE	Graph Sample and Aggregation	Mạng lấy mẫu và tổng hợp trên đồ thị
AI	Artificial Intelligence	Trí tuệ nhân tạo
GNN	Graph Neural Network	Mạng nơ-ron đồ thị
IoT	Internet of Thing	Internet vạn vật
GIN	Graph Isomorphism Network	Mạng đẳng cấu đồ thị
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
LSTM	Long Short Term Memory	Bộ nhớ dài hạn ngắn

CHƯƠNG 1: TỔNG QUAN VỀ CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ

1.1. Giới thiệu về mạng nơ-ron đồ thị

1.1.1. Bối cảnh ra đời và sự cần thiết của GNN

Trong thời kỳ bùng nổ của trí tuệ nhân tạo, các mô hình học sâu như mạng nơ-ron tích chập và mạng hồi tiếp đã chứng minh hiệu quả vượt trội trong việc xử lý dữ liệu có cấu trúc dạng lưới như hình ảnh, âm thanh hay chuỗi thời gian. Tuy nhiên, phần lớn dữ liệu trong thế giới thực lại không tồn tại dưới dạng lưới cố định mà có cấu trúc phi tuyến, phức tạp và mang tính liên kết - ví dụ như mạng xã hội, bản đồ giao thông, hệ thống protein hay quan hệ giữa các thực thể trong cơ sở tri thức. Các dạng dữ liệu này thường được biểu diễn dưới dạng đồ thị, trong đó các nút biểu thị thực thể và các cạnh thể hiện mối quan hệ giữa chúng.



Hình 1 : Tổng quan về công nghệ mạng nơ ron đồ thị

Trước sự hạn chế của các mô hình truyền thống trong việc học biểu diễn cho dữ liệu phi cấu trúc, mạng nơ-ron đồ thị (GNN) ra đời như một hướng tiếp cận mới nhằm tận dụng sức mạnh của học sâu cho dữ liệu dạng đồ thị. GNN cho phép các nút trong đồ thị trao đổi thông tin và cập nhật trạng thái dựa trên mối liên hệ với các nút láng giềng, từ đó hình thành biểu diễn đặc trưng toàn cục phản ánh cả nội dung của từng nút lẫn cấu trúc liên kết tổng thể của đồ thị.

Sự cần thiết của GNN thể hiện rõ qua việc ngày càng có nhiều lĩnh vực cần đến khả năng phân tích mối quan hệ phức tạp thay vì chỉ xử lý dữ liệu độc lập. Trong mạng

xã hội, GNN giúp dự đoán mối quan hệ bạn bè hoặc phát hiện cộng đồng; trong sinh học, GNN hỗ trợ dự đoán tương tác giữa protein; trong giao thông, GNN giúp dự báo luồng di chuyển giữa các vị trí; trong lĩnh vực khuyến nghị, GNN khai thác mối liên kết giữa người dùng và sản phẩm để đưa ra gợi ý chính xác hơn.

Nhờ khả năng mô hình hóa mạnh mẽ và linh hoạt, GNN đang trở thành một trong những công nghệ cốt lõi của trí tuệ nhân tạo hiện đại, đóng vai trò cầu nối giữa lý thuyết đồ thị truyền thống và học sâu trên dữ liệu phi cấu trúc, mở ra hướng phát triển mới cho nhiều bài toán trong khoa học và công nghiệp.

1.1.2. Khái niệm cơ bản về đồ thị trong học sâu

1.1.2.1. Các thành phần cơ bản

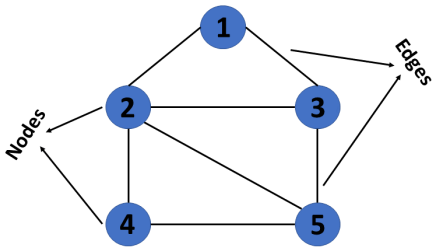
Trong một mạng nơ-ron đồ thị, dữ liệu đầu vào được mô hình hóa dưới dạng đồ thị (graph), bao gồm các nút (nodes) và cạnh (edges). Mỗi phần tử này đại diện cho một thực thể và mối quan hệ giữa các thực thể đó trong thế giới thực. Cấu trúc đồ thị cho phép mô hình học sâu khai thác không chỉ thông tin nội tại của từng phần tử mà còn cả cấu trúc kết nối giữa chúng - điều mà các mạng nơ-ron truyền thống khó thực hiện.

Nút (Node / Vertex): Là đơn vị cơ bản nhất trong đồ thị, biểu diễn cho một đối tượng hoặc thực thể trong hệ thống. Ví dụ, trong mạng xã hội, mỗi nút có thể là một người dùng; trong mạng phân tử, mỗi nút là một nguyên tử; còn trong mạng giao thông, nút có thể đại diện cho một giao lộ. Mỗi nút có thể mang theo một tập thuộc tính đặc trưng (feature vector) biểu diễn các thông tin định danh hoặc đặc điểm của thực thể, chẳng hạn như độ tuổi, vị trí địa lý, hoặc loại nguyên tử.

Cạnh (Edge): Là liên kết giữa hai nút, biểu diễn mối quan hệ hoặc sự tương tác giữa chúng. Cạnh có thể có hướng hoặc không có hướng, tùy vào tính chất của mối quan hệ. Ví dụ, trong mạng xã hội, một cạnh vô hướng có thể mô tả quan hệ “bạn bè”, trong khi cạnh có hướng thể hiện mối quan hệ “theo dõi” hoặc “ảnh hưởng”.

Thuộc tính cạnh (Edge Attributes): Cũng giống như nút, cạnh có thể mang các thuộc tính riêng mô tả mức độ hoặc tính chất của mối quan hệ, như cường độ tương tác, khoảng cách giữa hai điểm, hoặc xác suất xảy ra kết nối. Các thông tin này thường được

mã hóa thành các vector đặc trưng và được sử dụng trong quá trình huấn luyện mô hình GNN để giúp mạng hiểu rõ hơn ngữ cảnh của mối liên hệ.



Hình 2 : Các thành phần cơ bản của đồ thị

Biểu diễn toán học (khái niệm tổng quát): Một đồ thị thường được biểu diễn dưới dạng G = (V, E), trong đó V là tập hợp các nút và E là tập hợp các cạnh. Để lưu trữ thông tin một cách có hệ thống, người ta thường dùng ma trận kề, trong đó mỗi phần tử A_{ij} thể hiện sự tồn tại hoặc trọng số của liên kết giữa nút i và nút j. Tuy phần này mang tính toán học, nhưng ở mức khái niệm, cho phép GNN "nhìn thấy" toàn bộ cấu trúc kết nối và lan truyền thông tin hiệu quả giữa các nút.

Tóm lại, ba yếu tố - nút, cạnh và thuộc tính - là nền tảng cấu thành nên mọi mô hình GNN. Chúng quyết định cách mạng nơ-ron đồ thị học hỏi, biểu diễn và trích xuất thông tin từ dữ liệu có cấu trúc phức tạp.

1.1.2.2. Phân loại đồ thị

Tùy thuộc vào đặc điểm của dữ liệu và mối quan hệ giữa các thực thể, đồ thị có thể được phân loại theo nhiều tiêu chí khác nhau. Việc phân loại này giúp các mô hình mạng nơ-ron đồ thị (GNN) lựa chọn cơ chế lan truyền thông tin phù hợp, đảm bảo phản ánh chính xác bản chất và cấu trúc của hệ thống được mô hình hóa.

Đồ thị vô hướng là loại trong đó các cạnh không có hướng cụ thể, nghĩa là mối liên kết giữa hai nút được xem là tương hỗ. Khi nút A kết nối với nút B thì đồng thời B cũng kết nối với A. Loại đồ thị này thường được ứng dụng trong các bài toán phát hiện cộng đồng, phân tích cấu trúc mạng hoặc đo lường mức độ tương đồng giữa các nút.

Một ví dụ quen thuộc là mạng xã hội, nơi mối quan hệ "bạn bè" mang tính hai chiều - nếu hai người là bạn của nhau, kết nối đó tồn tại theo cả hai hướng.

Trái lại, đồ thị có hướng chứa các cạnh mang chiều xác định, phản ánh mối quan hệ một chiều giữa các nút. Một cạnh (u, v) thể hiện sự kết nối từ nút u đến nút v nhưng không đảm bảo chiều ngược lại. Dạng đồ thị này phổ biến trong các hệ thống khuyến nghị, mô hình mạng tri thức hoặc bài toán lan truyền thông tin. Ví dụ, trên nền tảng Twitter, người dùng A có thể "theo dõi" người dùng B mà không nhất thiết B phải theo dõi lại A, thể hiện rõ tính định hướng của mối liên kết.

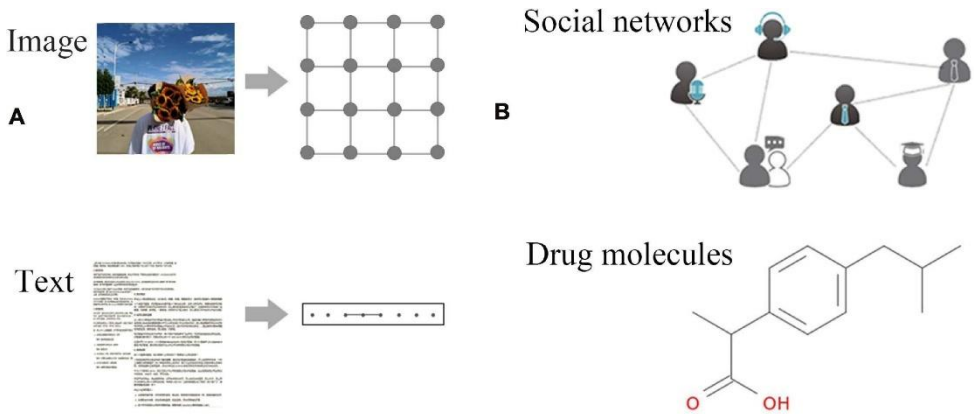
Về yếu tố trọng số, đồ thị có trọng số là loại trong đó mỗi cạnh được gán một giá trị thể hiện cường độ, chi phí hoặc độ quan trọng của mối quan hệ. Trọng số giúp mô hình GNN nhận biết và ưu tiên các kết nối có ảnh hưởng lớn hơn trong quá trình lan truyền đặc trưng. Ngược lại, đồ thị vô trọng số chỉ phản ánh sự tồn tại hay không tồn tại của một liên kết mà không kèm theo thông tin định lượng, thường được sử dụng trong các bài toán phân cụm hoặc phân tích cấu trúc mạng cơ bản.

Xét theo yếu tố thời gian, đồ thị tĩnh có cấu trúc cố định, các nút và cạnh không thay đổi. Đây là loại đồ thị được dùng phổ biến trong các nghiên cứu cơ bản về GNN, ví dụ như mô hình cấu trúc phân tử - nơi các nguyên tử và liên kết hóa học giữa chúng giữ nguyên theo thời gian. Trái lại, đồ thị động có thể thay đổi về số lượng nút, cạnh hoặc trọng số theo thời gian. Dạng này thường xuất hiện trong mạng xã hội hoặc mạng giao thông, nơi mối quan hệ hoặc điều kiện luôn biến động. Từ các dạng phân loại trên, có thể thấy rằng mỗi loại đồ thị mang đặc trưng và yêu cầu riêng, đòi hỏi mô hình GNN phải có khả năng thích ứng linh hoạt để nắm bắt được cấu trúc và quy luật đặc thù của dữ liệu mà mô hình xử lý.

1.1.3. Ứng dụng của mạng nơ-ron đồ thị trong thực tế

Mạng nơ-ron đồ thị (GNN) đã trở thành một trong những công nghệ tiên tiến nhất trong lĩnh vực học sâu hiện nay, nhờ khả năng xử lý hiệu quả các dạng dữ liệu có cấu trúc phức tạp. Trong khi các mạng nơ-ron truyền thống như CNN hay RNN chỉ hoạt động tốt trên dữ liệu có cấu trúc lưới như hình ảnh hay chuỗi thời gian, GNN lại có thể học được thông tin từ dữ liệu mà trong đó các phần tử tồn tại mối liên hệ phi tuyến và

không cố định. Nhờ đặc tính linh hoạt này, GNN đã được ứng dụng trong nhiều lĩnh vực thực tế, từ mạng xã hội, sinh học, hệ thống gợi ý cho tới giao thông thông minh và an ninh mạng.



Hình 3 : Các ứng dụng của mạng nơ-ron đồ thị

Trong lĩnh vực mạng xã hội, GNN đóng vai trò quan trọng trong việc phân tích hành vi người dùng và phát hiện các mối quan hệ tiềm ẩn. Mỗi người dùng được biểu diễn dưới dạng một nút, còn các tương tác như "bạn bè", "theo dõi" hay "thích" được mô hình hóa bằng các cạnh nối giữa các nút. Nhờ cơ chế lan truyền thông tin giữa các nút lân cận, GNN có thể học được bối cảnh xã hội và đặc điểm hành vi của từng người, từ đó được sử dụng cho các bài toán như dự đoán mối quan hệ mới, phân loại người dùng hoặc phát hiện cộng đồng. Khả năng học biểu diễn từ cấu trúc liên kết cho phép các mạng xã hội tối ưu hóa hệ thống đề xuất bạn bè, cải thiện hiệu suất quảng cáo và nâng cao mức độ tương tác của người dùng.

Trong sinh học tính toán và hóa học, GNN giúp mô hình hóa các phân tử và hợp chất dưới dạng đồ thị, trong đó các nút biểu diễn nguyên tử và các cạnh biểu diễn liên kết hóa học. Thông qua việc học biểu diễn cấu trúc phân tử, GNN có thể dự đoán các đặc tính vật lý, hóa học hoặc sinh học của hợp chất, chẳng hạn như độ hòa tan, khả năng phản ứng hay độc tính. Ứng dụng này đặc biệt có ý nghĩa trong quá trình phát hiện và thiết kế thuốc mới, khi mô hình có thể dự đoán hoạt tính sinh học của hàng triệu hợp chất mà không cần thử nghiệm thực tế, giúp tiết kiệm thời gian và chi phí nghiên cứu đáng kể.

Bên cạnh đó, trong các hệ thống gợi ý, GNN đã chứng minh hiệu quả vượt trội so với các phương pháp truyền thống. Dữ liệu người dùng và sản phẩm được mô hình hóa thành đồ thị, nơi GNN học biểu diễn đồng thời cho cả hai loại thực thể, dựa trên mối

quan hệ tương tác giữa chúng. Điều này giúp hệ thống khuyến nghị có thể nhận biết được các mối quan hệ gián tiếp, chẳng hạn như hai người dùng không có hành vi giống nhau nhưng có bạn chung hoặc quan tâm tới những sản phẩm tương tự. Nhờ vậy, các nền tảng như Amazon, YouTube hay Spotify có thể đưa ra những gợi ý mang tính cá nhân hóa và có độ chính xác cao hơn.

Một lĩnh vực ứng dụng khác là giao thông thông minh và IoT. Trong các hệ thống này, các nút có thể đại diện cho cảm biến, trạm thu thập dữ liệu hoặc giao lộ, còn các cạnh thể hiện luồng di chuyển hoặc kết nối vật lý giữa chúng. GNN được sử dụng để dự báo lưu lượng giao thông, phát hiện bất thường trong mạng cảm biến, hoặc tối ưu hóa luồng vận chuyển trong đô thị. Khả năng mô hình hóa quan hệ phụ thuộc không gian - thời gian của GNN giúp các hệ thống này trở nên thích ứng và hiệu quả hơn, đặc biệt trong bối cảnh dữ liệu biến động liên tục theo thời gian thực.

Trong lĩnh vực an ninh mạng và tài chính, GNN hỗ trợ phát hiện gian lận và nhận dạng hành vi bất thường. Khi các thực thể như tài khoản, giao dịch hay thiết bị mạng được biểu diễn thành đồ thị, GNN có thể học được các mẫu tương tác phức tạp giữa chúng, thay vì chỉ phân tích đặc trưng riêng lẻ. Việc nhận biết các mẫu quan hệ ẩn này cho phép phát hiện các giao dịch đáng ngờ, nhận diện tấn công mạng hoặc hành vi rửa tiền. Sức mạnh của GNN nằm ở khả năng phát hiện bất thường không chỉ từ hành vi riêng biệt mà còn từ cấu trúc liên kết của toàn bộ mạng lưới giao dịch hoặc kết nối.

Ngoài ra, GNN cũng được ứng dụng ngày càng nhiều trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP). Bằng cách biểu diễn câu hoặc đoạn văn dưới dạng đồ thị cú pháp và ngữ nghĩa, GNN có thể học được mối quan hệ phụ thuộc giữa các từ, giúp cải thiện độ chính xác của các bài toán như phân tích cảm xúc, trả lời câu hỏi hay dịch máy. Việc nắm bắt được ngữ cảnh sâu thông qua cấu trúc đồ thị cho phép mô hình hiểu được nghĩa toàn cục của câu thay vì chỉ dựa trên thứ tự tuyến tính của các từ.

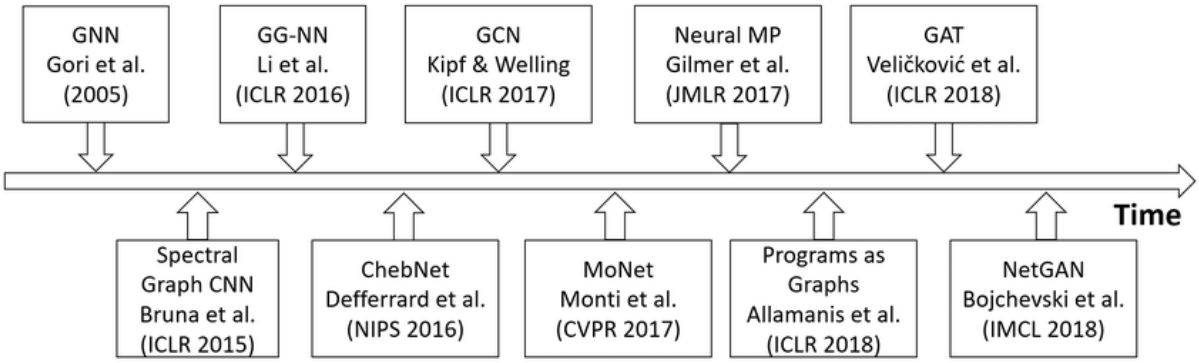
Tổng hợp lại, có thể thấy GNN đang đóng vai trò nền tảng trong nhiều lĩnh vực nghiên cứu và ứng dụng thực tế. Khả năng biểu diễn dữ liệu phi cấu trúc dưới dạng đồ thị, cùng với cơ chế lan truyền thông tin hiệu quả, giúp GNN vượt trội trong việc trích xuất tri thức từ các hệ thống phức tạp. Từ khoa học tự nhiên đến kỹ thuật, từ mạng xã

hội đến sinh học phân tử, GNN đang mở ra hướng đi mới trong việc kết nối giữa dữ liệu, tri thức và trí tuệ nhân tạo.

1.2. Lịch sử phát triển và các hướng nghiên cứu hiện nay

1.2.1. Lịch sử phát triển của GNN

Mạng nơ-ron đồ thị (GNN) là một trong những bước tiến quan trọng của trí tuệ nhân tạo hiện đại, xuất hiện nhằm giải quyết những hạn chế của các mô hình học sâu truyền thống khi làm việc với dữ liệu có cấu trúc phức tạp. Trước khi GNN ra đời, các mạng nơ-ron như CNN hay RNN đã đạt được nhiều thành công trong xử lý dữ liệu dạng lưới - chẳng hạn như ảnh hoặc chuỗi thời gian. Tuy nhiên, khi đối diện với các loại dữ liệu phi cấu trúc hoặc có mối quan hệ không cố định, chẳng hạn như mạng xã hội, phân tử hóa học, hay hệ thống giao thông, những mô hình này tỏ ra không còn phù hợp. Chính từ nhu cầu này, ý tưởng về mạng nơ-ron đồ thị đã được hình thành.



Hình 4 : Lịch sử phát tiển của GNN

Những nghiên cứu đầu tiên về GNN xuất hiện vào đầu những năm 2000. Pioneering work của Gori et al. (2005) và Scarselli et al. (2009) đã đặt nền móng lý thuyết cho GNN, khi họ đề xuất khái niệm về việc lan truyền thông tin giữa các nút thông qua các cạnh - một quá trình sau này được gọi là *message passing*. Tuy nhiên, do hạn chế về phần cứng và dữ liệu, các mô hình GNN thời kỳ này chủ yếu mang tính lý thuyết và chưa được triển khai rộng rãi.

Bước ngoặt lớn của GNN đến vào giai đoạn 2016 - 2018, khi mô hình GCN do Thomas Kipf và Max Welling (2016) giới thiệu đã chứng minh được hiệu quả vượt trội trong bài toán phân loại nút trên các tập dữ liệu đồ thị như Cora và Citeseer. GCN được

xem là phiên bản mở rộng của CNN lên miền đồ thị, cho phép thực hiện phép "tích chập" trên cấu trúc không gian phi Euclid. Từ đó, GNN nhanh chóng trở thành chủ đề nghiên cứu sôi nổi trong cộng đồng học sâu.

Cùng thời điểm đó, nhiều biến thể khác của GNN cũng được phát triển nhằm khắc phục những giới hạn của GCN. Năm 2017, GraphSAGE được giới thiệu với khả năng mở rộng cho đồ thị lớn bằng cách lấy mẫu cục bộ các nút lân cận. Năm 2018, GAT áp dụng cơ chế tự chú ý (*self-attention*) vào đồ thị, giúp mô hình tự động học trọng số khác nhau cho các nút lân cận thay vì coi tất cả như nhau. Đây là bước tiến quan trọng giúp GNN trở nên linh hoạt và chính xác hơn trong việc học biểu diễn đặc trưng.

Kể từ năm 2020 trở lại đây, GNN đã mở rộng ra nhiều hướng phát triển khác nhau: từ GNN động (Dynamic GNN) cho dữ liệu thời gian thực, đến GNN kết hợp Transformer, hay các mô hình khuếch tán nhằm mô phỏng quá trình lan truyền trong mạng. Các tập dữ liệu đồ thị khổng lồ và phần cứng GPU hiện đại đã tạo điều kiện để GNN được ứng dụng rộng rãi trong nhiều lĩnh vực như sinh học phân tử, phát hiện gian lận tài chính, phân tích mạng xã hội, và khuyến nghị thông minh.

Tóm lại, lịch sử phát triển của GNN là một quá trình tiến hóa liên tục - từ ý tưởng ban đầu mang tính khái niệm đến những mô hình học sâu tiên tiến có khả năng học biểu diễn mạnh mẽ trên dữ liệu đồ thị phức tạp. Sự phát triển này không chỉ phản ánh nỗ lực vượt qua giới hạn của học sâu truyền thống mà còn mở ra một kỷ nguyên mới trong việc xử lý dữ liệu phi cấu trúc trong thế giới thực.

1.2.2. Các hướng phát triển hiện nay

Trong quá trình tiến hóa của mạng nơ-ron đồ thị (GNN), nhiều hướng nghiên cứu mới đã được hình thành nhằm mở rộng khả năng biểu diễn và ứng dụng của mô hình. Các cải tiến này tập trung vào việc giúp GNN xử lý dữ liệu phức tạp hơn, quy mô lớn hơn, và có khả năng thích ứng với các thay đổi trong thời gian thực. Một số hướng phát triển nổi bật bao gồm GNN động, nơi cấu trúc đồ thị thay đổi theo thời gian; GNN phân cấp, giúp mô hình hóa các mối quan hệ đa tầng; các phương pháp học có giám sát, bán giám sát và không giám sát, giúp huấn luyện hiệu quả trong điều kiện dữ liệu nhãn hạn chế; và GNN kết hợp với mô hình sinh hoặc khuếch tán, nhằm mở rộng ứng dụng trong mô phỏng, sinh dữ liệu, và dự đoán tiến hóa mạng. Các hướng nghiên cứu này đang góp

phần đưa GNN trở thành một trong những công nghệ học sâu quan trọng nhất trong việc khai thác dữ liệu có cấu trúc phi Euclid.

1.2.2.1. GNN động (Dynamic GNN)

GNN động được phát triển nhằm giải quyết một hạn chế lớn của các mô hình GNN truyền thống - đó là giả định rằng cấu trúc đồ thị là cố định trong suốt quá trình học. Tuy nhiên, trong nhiều hệ thống thực tế như mạng xã hội, giao thông, hoặc mạng cảm biến, cấu trúc đồ thị thay đổi liên tục: các nút có thể xuất hiện, biến mất hoặc kết nối lại theo thời gian. Chính vì vậy, Dynamic GNN ra đời để mô hình hóa sự tiến hóa theo thời gian của đồ thị, giúp mô hình không chỉ hiểu được cấu trúc hiện tại mà còn nắm bắt được động lực học của toàn hệ thống.

Dynamic GNN kết hợp giữa học biểu diễn đồ thị và mô hình thời gian, thường sử dụng các cơ chế như RNN, LSTM, hoặc Transformer để ghi nhớ và cập nhật trạng thái của các nút theo chuỗi thời gian. Một số mô hình tiêu biểu gồm EvolveGCN, cho phép các tham số của mạng GCN thay đổi theo thời gian, hay TGAT (Temporal Graph Attention Network), mở rộng cơ chế chú ý sang miền thời gian để nắm bắt mối liên hệ tạm thời giữa các sự kiện.

Về cách tiếp cận, Dynamic GNN thường chia thành hai nhóm chính:

- **Discrete-time GNN**: xem dữ liệu đồ thị như chuỗi các ảnh chụp tại những thời điểm rời rạc, mỗi ảnh là một đồ thị khác nhau.
- **Continuous-time GNN**: mô hình hóa dữ liệu ở mức liên tục, theo dõi sự thay đổi của nút hoặc cạnh bằng các quá trình ngẫu nhiên hoặc hệ vi phân.

Ứng dụng của Dynamic GNN trải rộng từ dự đoán mối quan hệ tương lai trong mạng xã hội, phát hiện bất thường trong hệ thống IoT, đến mô hình hóa sự biến đổi cấu trúc phân tử theo thời gian. Tuy nhiên, các thách thức của hướng nghiên cứu này cũng không nhỏ - bao gồm khối lượng tính toán lớn, khó khăn trong việc huấn luyện mô hình dài hạn, và nhu cầu dữ liệu lịch sử phong phú để đảm bảo tính chính xác.

Tóm lại, GNN động đã mở rộng phạm vi ứng dụng của mạng nơ-ron đồ thị từ không gian tĩnh sang miền thời gian, giúp các mô hình trở nên thích ứng hơn với dữ liệu thực tế - nơi mà cấu trúc và mối quan hệ luôn biến đổi liên tục.

1.2.2.2. GNN phân cấp (Hierarchical / Pooling GNN)

Trong các mạng nơ-ron truyền thống như CNN, quá trình gộp đặc trưng giúp trích xuất các thông tin khái quát hơn, đồng thời giảm số lượng tham số và cải thiện khả năng khái quát hóa của mô hình. Ý tưởng tương tự cũng được áp dụng vào mạng nơ-ron đồ thị, hình thành nên khái niệm GNN phân cấp hay Hierarchical GNN. Mục tiêu của hướng nghiên cứu này là cho phép mô hình hiểu được cấu trúc đa tầng của đồ thị, thay vì chỉ dựa vào các kết nối cục bộ giữa các nút.

Trong GNN thông thường, việc lan truyền thông tin diễn ra ở mức nút, nơi mỗi nút thu thập và tổng hợp thông tin từ hàng xóm gần kề. Tuy nhiên, điều này chưa đủ để nắm bắt các mối quan hệ ở mức cao hơn, chẳng hạn như mối quan hệ giữa các nhóm nút hoặc cộng đồng trong đồ thị. Do đó, các mô hình GNN phân cấp đưa ra thêm các bước gộp (pooling) hoặc nén cấu trúc (coarsening), trong đó các nút có tính chất tương tự được gộp lại thành siêu nút. Quá trình này giúp mô hình có thể học được đặc trưng toàn cục và mở rộng khả năng xử lý với đồ thị quy mô lớn.

Một số phương pháp pooling nổi bật bao gồm:

- **DiffPool (Differentiable Pooling)**: cho phép mô hình học cách gộp các nút thông qua một ma trận phân cụm có thể huấn luyện được. DiffPool hoạt động như một lớp mạng con, giúp tạo ra biểu diễn gộp động thay vì dựa trên cấu trúc cố định.
- **TopKPooling**: lựa chọn một phần các nút có giá trị đặc trưng quan trọng nhất (theo một tiêu chí học được) và loại bỏ các nút ít liên quan hơn. Cách này giúp giảm số nút theo một tỷ lệ kiểm soát được, tương tự như max pooling trong CNN.
- **SAGPool (Self-Attention Graph Pooling)**: kết hợp cơ chế tự chú ý (*self-attention*) để xác định mức độ quan trọng của từng nút, từ đó chọn ra các nút đại diện có ảnh hưởng lớn nhất đến biểu diễn của toàn đồ thị.

Nhờ cơ chế phân cấp này, GNN có thể mô hình hóa cấu trúc đa tỷ lệ. Điều này đặc biệt hữu ích trong các bài toán như phân loại đồ thị, phân tích cấu trúc phân tử, dự đoán

tính chất hóa học, hay phân tích mạng xã hội, nơi mà mối quan hệ giữa các nhóm nút mang ý nghĩa lớn hơn từng nút riêng lẻ.

Tuy nhiên, GNN phân cấp cũng đối mặt với những thách thức trong quá trình huấn luyện. Việc gộp sai hoặc loại bỏ các nút quan trọng có thể làm mất thông tin cấu trúc. Do đó, nhiều nghiên cứu hiện nay tập trung vào việc cải thiện các cơ chế và kết hợp với chú ý hoặc cơ chế khuếch tán để tăng độ chính xác trong quá trình tổng hợp thông tin.

1.2.2.3. Học có giám sát, bán giám sát, không giám sát

Tương tự như các mô hình học sâu truyền thống, các mạng nơ-ron đồ thị (GNN) cũng có thể được huấn luyện theo nhiều mức độ giám sát khác nhau, bao gồm có giám sát, bán giám sát và không giám sát. Việc lựa chọn phương thức học phù hợp phụ thuộc vào mức độ sẵn có của dữ liệu nhãn và mục tiêu cụ thể của bài toán.

Học có giám sát (Supervised Learning) trong GNN thường được áp dụng khi tất cả hoặc phần lớn các nút (hoặc đồ thị) đều có nhãn. Mô hình sẽ học cách ánh xạ từ đặc trưng đầu vào đến nhãn đầu ra. Ví dụ điển hình là bài toán phân loại đồ thị hóa học hoặc dự đoán loại người dùng trong mạng xã hội, nơi mỗi nút hoặc mỗi đồ thị tương ứng với một lớp xác định. GCN và GAT là hai mô hình thường được dùng trong dạng học này vì khả năng học đặc trưng cục bộ tốt và tối ưu hóa dễ dàng bằng hàm mất mát chuẩn. Tuy nhiên, học có giám sát đòi hỏi số lượng nhãn lớn, điều này thường không khả thi trong các ứng dụng thực tế.

Để khắc phục hạn chế đó, học bán giám sát (Semi-Supervised Learning) được xem là hướng tiếp cận quan trọng trong GNN. Ở đây, chỉ một phần nhỏ nút có nhãn, trong khi phần lớn còn lại là dữ liệu chưa gán nhãn. GNN tận dụng cấu trúc đồ thị để lan truyền thông tin từ các nút có nhãn đến các nút không nhãn thông qua quá trình *message passing*. Mô hình GCN của Kipf và Welling (2017) là ví dụ kinh điển cho hướng học bán giám sát, khi mô hình có thể đạt hiệu quả cao dù chỉ sử dụng một lượng nhãn rất nhỏ. Phương pháp này đặc biệt hữu ích trong các mạng lớn như Cora, Citeseer, nơi việc gán nhãn thủ công rất tốn kém.

Bên cạnh đó, học không giám sát (Unsupervised Learning) trong GNN hướng đến việc học biểu diễn của các nút hoặc đồ thị mà không cần nhãn. Các mô hình này tối ưu

dựa trên mục tiêu tự định nghĩa, chẳng hạn như tối đa hóa độ tương đồng giữa các nút có liên kết và giảm tương đồng giữa các nút không liên kết. Một số phương pháp nổi bật là Graph Autoencoder (GAE), Deep Graph Infomax (DGI) và các kỹ thuật dựa trên học tương phản (contrastive learning). Mục tiêu của chúng là trích xuất ra biểu diễn ẩn giàu thông tin, có thể dùng cho các tác vụ khác như phân cụm hoặc phát hiện bất thường.

Nhìn chung, ba hướng tiếp cận này bổ sung cho nhau trong việc mở rộng khả năng ứng dụng. Học có giám sát mang lại độ chính xác cao khi có đủ dữ liệu, học bán giám sát giúp tận dụng tốt cấu trúc đồ thị trong môi trường thiếu nhãn, còn học không giám sát mở đường cho việc khai thác dữ liệu quy mô lớn không có nhãn sẵn. Trong nghiên cứu hiện nay, các mô hình kết hợp cũng đang được quan tâm mạnh mẽ, khi chúng có thể tự thích ứng giữa các điều kiện dữ liệu khác nhau để đạt hiệu năng tối ưu.

1.2.3. Thách thức trong nghiên cứu và ứng dụng

Mặc dù mạng nơ-ron đồ thị (GNN) đã chứng minh được tiềm năng vượt trội trong nhiều lĩnh vực, song quá trình nghiên cứu và ứng dụng công nghệ này vẫn đối mặt với hàng loạt thách thức khoa học và kỹ thuật cần được giải quyết. Các thách thức này không chỉ đến từ bản chất phức tạp của dữ liệu đồ thị mà còn từ những giới hạn trong khả năng biểu diễn, mở rộng và giải thích của mô hình.

Một trong những khó khăn nổi bật là vấn đề quá mượt (over-smoothing), xảy ra khi số tầng mạng tăng lên khiến biểu diễn của các nút trở nên gần như đồng nhất. Khi đó, mô hình mất khả năng phân biệt các nút khác nhau, làm giảm hiệu quả trong các tác vụ phân loại hay dự đoán. Việc cân bằng giữa chiều sâu của mạng và khả năng duy trì tính đa dạng đặc trưng là vấn đề được quan tâm trong hầu hết các nghiên cứu hiện nay.

Tiếp theo là vấn đề mở rộng, đặc biệt khi áp dụng GNN trên các đồ thị cực lớn như mạng xã hội toàn cầu, đồ thị tri thức hay mạng protein. Quá trình lan truyền thông tin trong GNN đòi hỏi việc truy cập và tổng hợp đặc trưng từ hàng triệu nút và cạnh, dẫn đến chi phí tính toán và bộ nhớ khổng lồ. Các kỹ thuật như lấy mẫu, mini-batch training hay phân tán tính toán (distributed training) đã được đề xuất để giảm tải, song vẫn chưa có giải pháp nào tối ưu hoàn toàn cho mọi loại dữ liệu đồ thị.

Một thách thức khác là khả năng diễn giải mô hình. Do cấu trúc của GNN thường phức tạp, việc hiểu rõ lý do mô hình đưa ra dự đoán nào đó vẫn là vấn đề nan giải. Trong các ứng dụng quan trọng như y sinh học, tài chính hay an ninh mạng, việc mô hình "đúng" là chưa đủ - mà cần giải thích được tại sao mô hình đúng. Các hướng nghiên cứu mới về graph explainability đang nỗ lực tạo ra các phương pháp giúp trực quan hóa mức độ đóng góp của từng nút hoặc cạnh trong quá trình suy luận.

Bên cạnh đó, tính không đồng nhất của dữ liệu đồ thị cũng đặt ra yêu cầu cao cho GNN. Trong thực tế, nhiều đồ thị chứa nhiều loại nút và cạnh khác nhau, như trong mạng tri thức hoặc mạng thương mại điện tử. Việc biểu diễn và xử lý hiệu quả loại dữ liệu này đòi hỏi các mô hình Heterogeneous GNN (HGNN) hoặc Relational GNN, nhưng chúng lại làm tăng đáng kể độ phức tạp và chi phí huấn luyện.

Ngoài ra, tính động của đồ thị cũng là một thách thức lớn. Trong nhiều hệ thống thực tế như mạng xã hội hoặc giao thông, cấu trúc đồ thị không cố định mà thay đổi liên tục theo thời gian. Do đó, việc cập nhật thông tin mà không cần huấn luyện lại toàn bộ mô hình là một hướng nghiên cứu còn nhiều tiềm năng nhưng cũng đầy khó khăn.

Cuối cùng, về mặt ứng dụng, vấn đề dữ liệu nhãn hạn chế, nhiễu trong thông tin cạnh, và độ tin cậy của mô hình trong môi trường thực tế vẫn là những rào cản quan trọng. Dữ liệu đồ thị thường được thu thập từ các nguồn không đồng nhất, dễ bị sai lệch hoặc thiếu sót, khiến việc huấn luyện mô hình gặp khó khăn. Đồng thời, khi triển khai trong thực tế, GNN cần phải đảm bảo khả năng tổng quát hóa và ổn định trước biến động dữ liệu, điều mà các mô hình hiện nay vẫn chưa đáp ứng hoàn toàn.

Tổng thể, có thể thấy rằng mặc dù GNN đã đạt được nhiều thành tựu ấn tượng, song con đường hướng tới một mô hình GNN toàn diện, hiệu quả, dễ mở rộng và dễ giải thích vẫn còn nhiều thách thức. Chính những thách thức này cũng tạo động lực mạnh mẽ cho các hướng nghiên cứu mới, giúp lĩnh vực mạng nơ-ron đồ thị tiếp tục phát triển và hoàn thiện trong tương lai.

1.3. Kết luận

Chương này đã trình bày tổng quan về công nghệ mạng nơ-ron đồ thị (GNN), bao gồm nền tảng hình thành, các khái niệm cơ bản về đồ thị trong học sâu, cùng những lĩnh

vực ứng dụng tiêu biểu. Ngoài ra, phần nội dung cũng đã khái quát quá trình phát triển của GNN, các hướng nghiên cứu hiện nay và những thách thức đang được quan tâm trong cộng đồng khoa học. Qua tổng hợp các kết quả nghiên cứu, có thể nhận thấy rằng GNN là một hướng tiếp cận tiềm năng trong việc xử lý và học biểu diễn dữ liệu có cấu trúc phức tạp. Việc kết hợp giữa lý thuyết đồ thị và các kỹ thuật học sâu đã mở ra khả năng khai thác hiệu quả các mối quan hệ phi tuyến giữa các thực thể, vượt ra ngoài giới hạn của những mô hình học truyền thống.

Bên cạnh tiềm năng đó, công nghệ GNN vẫn đối mặt với nhiều vấn đề nghiên cứu mở, như khả năng mở rộng trên đồ thị quy mô lớn, hiện tượng làm mượt quá mức (over-smoothing), cũng như yêu cầu cao về tài nguyên tính toán. Do đó, việc tiếp tục nghiên cứu tối ưu hóa cơ chế lan truyền thông tin, cải tiến cấu trúc mô hình và mở rộng phạm vi ứng dụng của GNN trong thực tiễn là những định hướng quan trọng trong giai đoạn sắp tới. Những nội dung của chương này đóng vai trò nền tảng cho các chương tiếp theo, trong đó các nguyên lý lý thuyết và mô hình cụ thể của mạng nơ-ron đồ thị sẽ được phân tích chuyên sâu hơn.

CHƯƠNG 2: CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ

2.1. Cơ sở lý thuyết về mạng nơ-ron đồ thị

2.1.1. Biểu diễn toán học của đồ thị

Trong học sâu trên cấu trúc đồ thị, việc mô hình hóa dữ liệu dưới dạng toán học là bước nền tảng, đóng vai trò then chốt giúp các mạng nơ-ron đồ thị có thể khai thác, xử lý và học được các đặc trưng tiềm ẩn của dữ liệu. Một đồ thị tổng quát được ký hiệu là G = (V, E), trong đó V = {v_1, v_2, ..., v_n} biểu diễn tập các nút và $E \subseteq V \times V$ là tập các cạnh thể hiện mối quan hệ giữa các nút. Mỗi nút v_i có thể mang một vector đặc trưng $x_i \in \mathbb{R}^d$, mô tả các thuộc tính hoặc thông tin ban đầu của nút như đặc trưng hình học, ngữ nghĩa hay tín hiệu cảm biến.

Cấu trúc liên kết giữa các nút trong đồ thị thường được mô tả bằng ma trận kề $A \in \mathbb{R}^{N \times N}$, trong đó phần tử A_{ij} nhận giá trị 1 nếu tồn tại cạnh nối giữa nút v_i và v_j, và bằng 0 trong trường hợp ngược lại. Đối với đồ thị có trọng số, giá trị của A_{ij} phản ánh cường độ hoặc mức độ quan hệ giữa hai nút, thay vì chỉ đơn thuần biểu thị sự tồn tại của cạnh. Trong đồ thị vô hướng, ma trận này đối xứng ($A = A^T$), trong khi ở đồ thị có hướng, tính đối xứng này không còn được đảm bảo. Nhờ vào ma trận kề, toàn bộ cấu trúc đồ thị có thể được biểu diễn một cách đầy đủ và thuận tiện cho các phép toán đại số trong huấn luyện mô hình.

Bên cạnh cấu trúc liên kết, mỗi nút trong đồ thị thường được gắn với một vector đặc trưng đầu vào. Tập hợp các vector này được biểu diễn bằng ma trận đặc trưng $X \in \mathbb{R}^{N \times d}$, trong đó hàng thứ i tương ứng với vector đặc trưng x_i^T của nút v_i. Ma trận này có vai trò tương tự như dữ liệu đầu vào trong mạng nơ-ron truyền thống, cung cấp thông tin mô tả cho từng nút để quá trình lan truyền và học biểu diễn diễn ra hiệu quả. Trong bối cảnh ứng dụng, các đặc trưng này có thể là thuộc tính của người dùng trong mạng xã hội, thông tin hóa học của nguyên tử trong phân tử, hay ngữ cảnh của từ trong bài toán xử lý ngôn ngữ tự nhiên.

Để phục vụ các phép biến đổi tín hiệu và tích chập trên đồ thị, người ta còn sử dụng ma trận bậc và ma trận Laplacian. Ma trận bậc D là ma trận đường chéo trong đó phần tử D_{ii} biểu diễn số lượng (hoặc tổng trọng số) các cạnh nối với nút v_i. Từ hai ma

trận A và D, ma trận Laplacian được định nghĩa là L = D − A. Cấu trúc này đóng vai trò quan trọng trong lý thuyết phổ (spectral graph theory), cho phép mô tả các phép biến đổi tín hiệu trên đồ thị dưới góc nhìn của giá trị riêng và vector riêng. Trong nhiều mô hình hiện đại như GCN, người ta thường sử dụng dạng Laplacian chuẩn hóa $L_{norm} = I - D^{-1/2}AD^{-1/2}$, nhằm đảm bảo quá trình lan truyền thông tin được ổn định và không bị chi phối bởi các nút có bậc lớn.

Tổng thể, việc biểu diễn đồ thị bằng các ma trận như A, X và L giúp định hình cơ sở toán học để mô tả cách dữ liệu lan truyền và học biểu diễn trong không gian phi Euclid. Những biểu diễn này không chỉ cho phép các mô hình GNN xử lý thông tin cấu trúc một cách tự nhiên mà còn tạo điều kiện cho việc xây dựng các cơ chế lan truyền thông tin - nền tảng của phần tiếp theo, nơi mạng học cách tổng hợp và cập nhật đặc trưng giữa các nút láng giềng.

2.1.2. Cơ chế lan truyền thông tin trên đồ thị

Cơ chế lan truyền thông tin (Message Passing) là nền tảng của hầu hết các mô hình mạng nơ-ron đồ thị hiện nay. Khác với các mạng nơ-ron truyền thống chỉ xử lý dữ liệu dạng vector hoặc ma trận có cấu trúc cố định, GNN hoạt động trên dữ liệu đồ thị - nơi các nút có mối quan hệ liên kết phức tạp. Vì vậy, việc cập nhật biểu diễn của từng nút không thể thực hiện độc lập mà phải dựa vào thông tin từ các nút lân cận. Cơ chế *message passing* được thiết kế nhằm mô phỏng quá trình này, cho phép các nút "trao đổi" thông tin với nhau thông qua các cạnh.

Trong một vòng lan truyền, mỗi nút v_i thu thập thông tin từ các nút láng giềng $\mathcal{N}(i)$ của nút, kết hợp với thông tin hiện tại của chính nút đó để hình thành biểu diễn đặc trưng mới. Quá trình này có thể được mô tả tổng quát bằng công thức:

$$h_i^{(k)} = UPDATE^{(k)}\left(h_i^{(k-1)}, AGGREGATE^{(k)}\left(\{h_j^{(k-1)} | j \in \mathcal{N}(i)\}\right)\right)$$

Trong đó:

- $h_i^{(k)}$ là vector biểu diễn của nút i tại tầng lan truyền thứ k.
- AGGREGATE là hàm tổng hợp thông tin từ các nút láng giềng.

- UPDATE là hàm cập nhật biểu diễn của nút dựa trên thông tin tổng hợp và trạng thái trước đó.

Quá trình *message passing* thường được chia thành ba giai đoạn chính. Thứ nhất, giai đoạn *Aggregation* chịu trách nhiệm thu thập đặc trưng của các nút lân cận. Hàm tổng hợp phải đảm bảo tính bất biến đối với thứ tự các láng giềng, thường được xây dựng dựa trên các phép toán như tổng, trung bình, hay lấy giá trị lớn nhất. Trong một số mô hình như GraphSAGE hoặc GAT, hàm này có thể được thay thế bằng cơ chế chú ý để gán trọng số khác nhau cho từng láng giềng, giúp mạng học được tầm quan trọng tương đối giữa các nút.

Thứ hai, giai đoạn *Combination* (hoặc *Transformation*) thực hiện việc kết hợp thông tin thu thập được với đặc trưng của chính nút trung tâm. Thông thường, đây là bước tuyến tính thông qua một ma trận trọng số huấn luyện được, có thể kèm theo hàm kích hoạt phi tuyến như ReLU. Giai đoạn này giúp biến đổi không gian đặc trưng, cho phép mô hình học được các biểu diễn trừu tượng hơn từ dữ liệu đầu vào.

Thứ ba, giai đoạn *Update* đảm nhiệm việc tính toán biểu diễn mới của nút sau khi đã kết hợp thông tin. Một số mô hình sử dụng phép cộng, nối giữa đặc trưng cũ và mới. Trong khi các mô hình tiên tiến hơn, ví dụ như mô hình Gated Graph Neural Network (GGNN), sử dụng cơ chế gate tương tự như trong LSTM để điều chỉnh lượng thông tin được cập nhật.

Quá trình lan truyền này được lặp lại qua nhiều tầng (layers), mỗi tầng mở rộng phạm vi thông tin mà nút có thể tiếp cận - hay nói cách khác, sau k tầng lan truyền, biểu diễn của một nút chứa thông tin từ các nút nằm trong bán kính k-bậc láng giềng. Tuy nhiên, việc tăng số tầng quá lớn có thể dẫn đến hiện tượng *over-smoothing*, khiến các biểu diễn của các nút dần trở nên đồng nhất và mất khả năng phân biệt. Do đó, việc xác định số tầng lan truyền tối ưu là yếu tố quan trọng trong quá trình thiết kế mô hình GNN.

Về bản chất, cơ chế *message passing* cho phép GNN học được cấu trúc và mối quan hệ trong đồ thị thông qua quá trình truyền tín hiệu giữa các nút. Cơ chế này không chỉ giúp khai thác hiệu quả thông tin cục bộ, mà còn duy trì được khả năng mở rộng cho các đồ thị có kích thước lớn và cấu trúc phức tạp. Nhờ vậy, các mô hình GNN có thể áp

dụng cho nhiều bài toán khác nhau, từ phân loại nút, dự đoán cạnh đến học biểu diễn toàn đồ thị.

2.1.3. Kiến trúc tổng quát của mô hình GNN

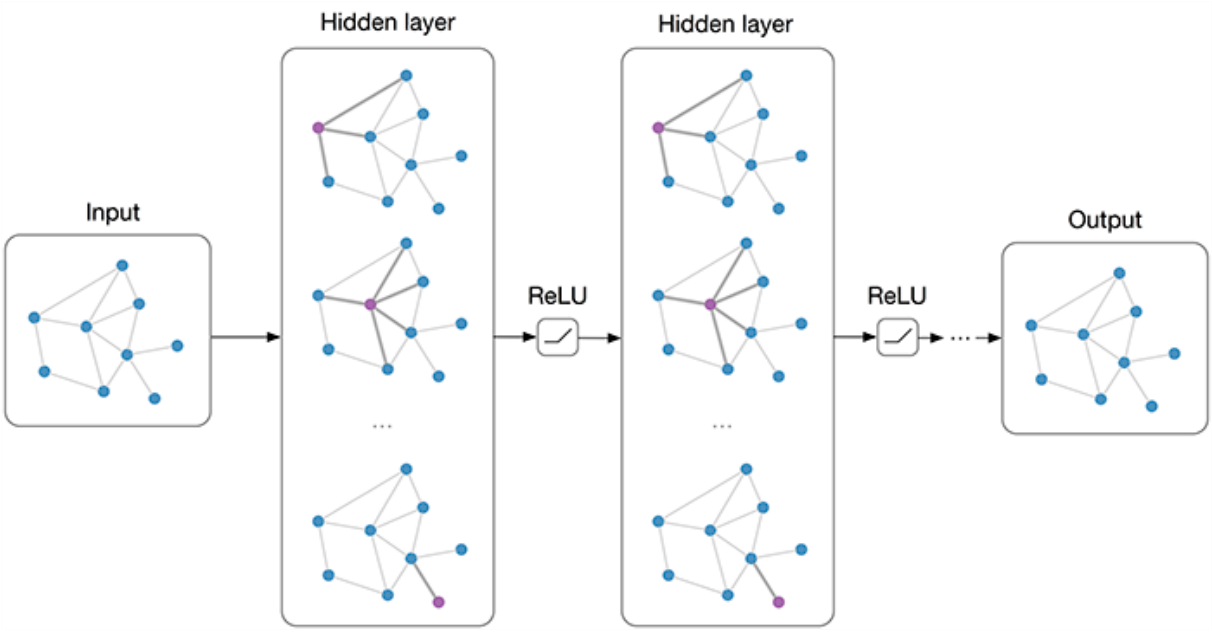
Kiến trúc tổng quát của một mô hình Graph Neural Network (GNN) được xây dựng dựa trên ba thành phần chính: lớp đầu vào, các lớp lan truyền thông tin và lớp đầu ra. Mỗi thành phần đảm nhận một vai trò cụ thể trong quá trình học biểu diễn và suy luận trên cấu trúc đồ thị.

Ở lớp đầu vào, mô hình tiếp nhận dữ liệu ban đầu gồm ma trận đặc trưng của các nút và ma trận kề mô tả cấu trúc kết nối giữa các nút. Những thông tin này cung cấp nền tảng để mô hình hiểu được cả nội dung và mối quan hệ giữa các thực thể trong đồ thị. Tiếp theo là các lớp lan truyền thông tin, được xem là phần cốt lõi của GNN. Tại đây, mỗi nút không chỉ học từ đặc trưng của chính nút đó mà còn từ các nút láng giềng thông qua cơ chế tổng hợp và cập nhật đã được trình bày ở mục trước. Mỗi lớp như vậy mở rộng "phạm vi nhận thức" của nút, giúp mô hình dần nắm bắt thông tin từ các vùng xa hơn trong đồ thị. Sau cùng, lớp đầu ra đảm nhận nhiệm vụ tổng hợp thông tin đã học được để đưa ra kết quả dự đoán phù hợp với bài toán đặt ra. Với phân loại nút, đầu ra là vector xác suất cho từng nút; còn với phân loại đồ thị, đặc trưng của toàn bộ các nút sẽ được gom lại thông qua các phép tổng hợp toàn cục nhằm tạo ra biểu diễn duy nhất cho toàn đồ thị.

Tổng thể, kiến trúc của GNN tuân theo quy trình: tiếp nhận đặc trưng - lan truyền thông tin - tổng hợp và suy luận. Cách tổ chức này cho phép mô hình vừa nắm bắt được thông tin cục bộ, vừa học được mối quan hệ toàn cục trong đồ thị, tạo nên khả năng biểu diễn mạnh mẽ và linh hoạt cho nhiều loại bài toán khác nhau.

2.2. Các mô hình mạng nơ-ron đồ thị phổ biến

2.2.1. Graph Convolutional Network (GCN)



Hình 5 : Mô hình GCN

Graph Convolutional Network (GCN) là một trong những mô hình nền tảng và có ảnh hưởng sâu rộng nhất trong lĩnh vực mạng nơ-ron đồ thị. Ý tưởng chính của GCN là mở rộng phép tích chập - vốn quen thuộc trong mạng nơ-ron tích chập (CNN) trên dữ liệu dạng lưới - sang miền dữ liệu có cấu trúc phi Euclid như đồ thị. Trong CNN truyền thống, phép tích chập được thực hiện bằng cách trượt một kernel cố định trên ảnh để trích xuất đặc trưng cục bộ, nhưng trong đồ thị, các nút không được sắp xếp theo dạng lưới nên việc định nghĩa "láng giềng" và "kích thước kernel" cần được khái quát lại.

GCN giải quyết vấn đề này bằng cách biểu diễn phép tích chập thông qua lan truyền đặc trưng giữa các nút kết nối trong đồ thị. Mỗi nút sẽ cập nhật biểu diễn của mình bằng cách tổng hợp thông tin từ các nút láng giềng, có thể coi đây là phép tích chập theo miền phổ (spectral domain) hoặc miền không gian (spatial domain) tùy cách định nghĩa.

2.2.1.1. Nguyên lý tích chập trên đồ thị

Khác với dữ liệu dạng lưới như hình ảnh hay chuỗi thời gian, dữ liệu đồ thị có cấu trúc phi Euclid, khiến việc áp dụng trực tiếp phép tích chập truyền thống trở nên không khả thi. Để khái quát phép tích chập sang miền đồ thị, các nhà nghiên cứu đã xây dựng khái niệm tích chập trên đồ thị, trong đó quá trình xử lý đặc trưng được thực hiện dựa trên cấu trúc kết nối giữa các nút thay vì vị trí không gian cố định.

Về mặt lý thuyết, tích chập trên đồ thị có thể được định nghĩa trong miền phổ thông qua biến đổi Fourier trên đồ thị. Cho đồ thị G = (V, E) với ma trận kề A và ma trận bậc D, ta xây dựng ma trận Laplacian L = D − A. Khi thực hiện phân rã riêng trị $L = U\Lambda U^T$, phép tích chập giữa tín hiệu x và bộ lọc phổ g_θ được biểu diễn như sau:

$$g_\theta \times x = U g_\theta(\Lambda) U^T x$$

Trong biểu thức này, U đóng vai trò tương tự như ma trận biến đổi Fourier, và $g_\theta(\Lambda)$ biểu diễn bộ lọc trong miền phổ. Tuy nhiên, việc tính toán phân rã ma trận riêng L có độ phức tạp cao O(n^3), nên cách tiếp cận này khó áp dụng cho các đồ thị lớn. Để khắc phục hạn chế đó, Kipf và Welling (2017) đã đề xuất một dạng xấp xỉ bậc thấp của phép lọc phổ bằng cách thêm cạnh tự nối (self-loop) và chuẩn hóa lại ma trận kề, đưa đến công thức lan truyền đặc trưng được sử dụng rộng rãi trong mô hình GCN:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

Trong đó:

- $\tilde{A} = A + I$ là ma trận kề có thêm cạnh tự nối, giúp mỗi nút giữ lại thông tin của chính nút đó.
- $\tilde{D}$ là ma trận bậc tương ứng của $\tilde{A}$.
- $H^{(l)}$ là đầu ra của tầng thứ l, với $H^{(0)} = X$ là ma trận đặc trưng đầu vào.
- $W^{(l)}$ là ma trận trọng số huấn luyện được.
- $\sigma(\cdot)$ là hàm kích hoạt phi tuyến (ví dụ: ReLU).

Về bản chất, phép nhân $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}$ thể hiện quá trình truyền và tổng hợp thông tin từ các nút lân cận - tương đương với quá trình tích chập trong mạng CNN, nhưng được điều chỉnh phù hợp với cấu trúc đồ thị. Kết hợp với phép biến đổi tuyến tính $W^{(l)}$ và hàm phi tuyến, mô hình GCN có khả năng học biểu diễn nút hiệu quả, đồng thời đảm bảo tính ổn định khi lan truyền qua nhiều tầng

2.2.1.2. Quá trình lan truyền thông tin

Trong mạng GCN, quá trình lan truyền thông tin được hiểu là cách mà đặc trưng của từng đỉnh trong đồ thị được cập nhật dựa trên thông tin từ các đỉnh lân cận. Ở mỗi

lớp của mô hình, mỗi đỉnh sẽ "nhận thông điệp" từ các đỉnh kề với đỉnh đang được xets, tổng hợp lại, rồi kết hợp với thông tin hiện có để tạo ra đặc trưng mới. Nhờ vậy, biểu diễn của một đỉnh không còn chỉ phản ánh thuộc tính riêng của đỉnh đó, mà còn thể hiện mối quan hệ cấu trúc trong toàn bộ mạng.

Cụ thể, quá trình lan truyền thông tin trong GCN có thể hình dung qua ba giai đoạn:

- **Thu thập thông tin:** Mỗi đỉnh thu nhận đặc trưng từ các đỉnh lân cận. Ví dụ, trong mạng xã hội, người dùng có thể "nhận" thông tin từ bạn bè của họ.
- **Kết hợp thông tin:** Thông tin thu được sẽ được tổng hợp cùng đặc trưng hiện tại của đỉnh, có thể qua phép cộng, trung bình hoặc nhân ma trận trọng số.
- **Kích hoạt phi tuyến:** Kết quả tổng hợp được đưa qua hàm phi tuyến (thường là ReLU) để tăng khả năng biểu diễn và học đặc trưng phức tạp hơn.

Quá trình này diễn ra đồng thời cho toàn bộ các đỉnh trong đồ thị, tạo nên một bước lan truyền thông tin duy nhất. Khi chồng nhiều lớp GCN liên tiếp, phạm vi ảnh hưởng của mỗi đỉnh sẽ mở rộng dần ra các tầng lân cận xa hơn. Ví dụ, sau hai lớp GCN, mỗi đỉnh có thể nắm bắt thông tin của các đỉnh cách đỉnh đó hai bước trong đồ thị.

Điều này giúp GCN có khả năng học được bối cảnh cục bộ và cấu trúc toàn cục trong mạng - một đặc điểm quan trọng khiến mô hình này vượt trội hơn các mạng nơ-ron truyền thống khi xử lý dữ liệu dạng đồ thị. Tuy nhiên, số lượng lớp cũng cần được lựa chọn phù hợp: quá ít lớp khiến mô hình không học được đủ ngữ cảnh, trong khi quá nhiều lớp có thể dẫn đến hiện tượng "over-smoothing", làm cho các biểu diễn của các đỉnh trở nên quá giống nhau.

2.2.1.3. Ứng dụng và hạn chế của GCN

Mạng tích chập đồ thị (GCN) đã trở thành một trong những mô hình cốt lõi trong lĩnh vực học sâu trên đồ thị nhờ khả năng khai thác cấu trúc liên kết giữa các nút và lan truyền thông tin hiệu quả trong không gian đồ thị. Với ưu thế trong việc biểu diễn dữ liệu có quan hệ phức tạp, GCN được ứng dụng rộng rãi trong nhiều lĩnh vực nghiên cứu và thực tiễn. Một trong những hướng ứng dụng phổ biến nhất là phân loại nút, nơi mô hình có thể dự đoán nhãn của các nút trong mạng xã hội, xác định loại protein trong

mạng sinh học hoặc phân loại bài báo trong mạng trích dẫn học thuật. Ngoài ra, GCN còn được sử dụng trong dự đoán liên kết, giúp xác định khả năng hình thành kết nối mới giữa hai nút - chẳng hạn như dự đoán mối quan hệ giữa người dùng trong hệ thống gợi ý hoặc dự đoán tương tác giữa các phân tử trong hóa sinh học.

Không chỉ dừng lại ở mức độ nút, GCN còn được mở rộng cho bài toán phân loại đồ thị, trong đó mô hình học cách biểu diễn toàn bộ đồ thị như một thực thể duy nhất. Điều này đặc biệt hữu ích trong các ứng dụng như nhận dạng cấu trúc phân tử, phân loại mạng lưới điện hoặc phát hiện bất thường trong dữ liệu mạng máy tính. Bên cạnh đó, GCN đã được áp dụng thành công trong các bài toán phức tạp liên quan đến dữ liệu không gian - thời gian, chẳng hạn như dự đoán lưu lượng giao thông, chất lượng không khí, hoặc nhu cầu năng lượng tại các khu vực đô thị. Nhờ khả năng xử lý mối quan hệ giữa các thực thể trong không gian, GCN đã chứng minh tính hiệu quả vượt trội so với các mô hình học sâu truyền thống vốn giả định dữ liệu có cấu trúc Euclid.

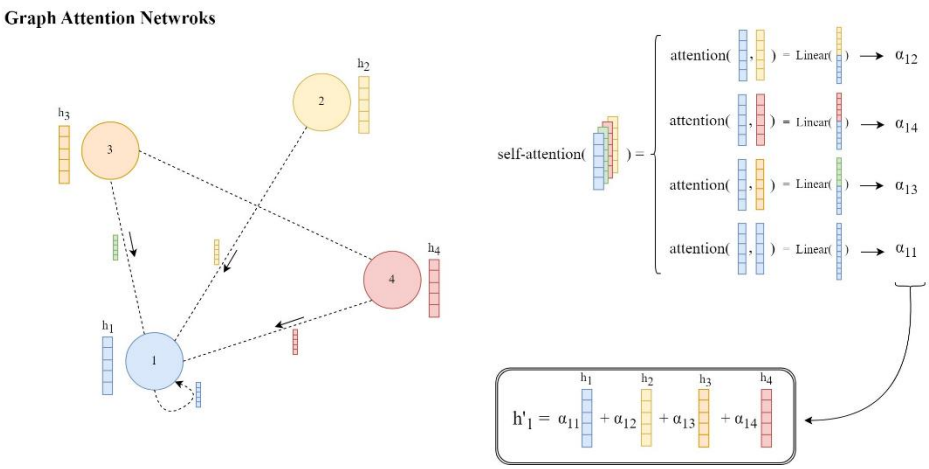
Tuy nhiên, bên cạnh những ưu điểm nổi bật, GCN vẫn tồn tại nhiều hạn chế cần được khắc phục. Vấn đề mở rộng là một trong những thách thức chính, khi việc xử lý toàn bộ ma trận kề trở nên tốn kém về bộ nhớ và thời gian tính toán đối với các đồ thị có quy mô lớn. Ngoài ra, mô hình còn gặp phải hiện tượng “over-smoothing”, khi số lượng lớp tích chập tăng lên khiến các biểu diễn nút trở nên quá đồng nhất, làm giảm khả năng phân biệt giữa các nút khác nhau. Một hạn chế khác là sự phụ thuộc mạnh vào cấu trúc đồ thị đầu vào; nếu đồ thị chứa nhiều nhiễu hoặc không phản ánh đúng quan hệ thực tế giữa các nút, hiệu quả huấn luyện sẽ bị suy giảm đáng kể. Bên cạnh đó, việc giải thích cơ chế hoạt động của GCN và điều chỉnh các tham số của mô hình vẫn là vấn đề khó khăn, do bản chất phi tuyến và sự lan truyền thông tin qua nhiều tầng trừu tượng.

Tổng thể, GCN là nền tảng quan trọng mở đường cho sự phát triển của nhiều biến thể tiên tiến hơn như GAT hay GraphSAGE, vốn được thiết kế nhằm khắc phục những hạn chế của mô hình gốc. Những cải tiến này đã giúp GNN tiếp cận được các bài toán có quy mô lớn và phức tạp hơn, đồng thời nâng cao khả năng học biểu diễn dữ liệu trong các hệ thống có cấu trúc đồ thị phong phú và động.

2.2.2. Graph Attention Network

Graph Attention Network (GAT) là một mô hình tiên tiến trong họ mạng nơ-ron đồ thị, được phát triển nhằm khắc phục hạn chế của các mô hình như GCN trong việc xử lý các nút láng giềng có mức độ quan trọng khác nhau. Thay vì gán trọng số đồng đều cho tất cả các kết nối, GAT áp dụng cơ chế attention - cho phép mạng tự động học cách "chú ý" đến những nút có ảnh hưởng lớn hơn trong quá trình lan truyền thông tin. Mỗi nút tính toán hệ số chú ý (attention coefficient) với từng láng giềng của nút đó, từ đó tổng hợp thông tin có trọng số phù hợp. Cách tiếp cận này giúp GAT tăng khả năng biểu diễn, đặc biệt trong các đồ thị không đồng nhất hoặc có cấu trúc phức tạp, đồng thời loại bỏ sự phụ thuộc vào cấu trúc cố định của đồ thị hoặc phép biến đổi Laplacian như trong GCN.

Bên cạnh đó, GAT còn mở rộng khả năng học biểu diễn nhờ cơ chế multi-head attention, trong đó nhiều "đầu chú ý" song song được huấn luyện để trích xuất các khía cạnh khác nhau của thông tin từ các nút láng giềng. Việc kết hợp nhiều đầu chú ý không chỉ giúp mô hình ổn định hơn trong quá trình huấn luyện mà còn tăng cường khả năng nắm bắt các mối quan hệ phức tạp trong đồ thị. Nhờ tính linh hoạt và hiệu quả này, GAT đã được ứng dụng rộng rãi trong nhiều lĩnh vực như phân loại nút, dự đoán liên kết và hệ thống gợi ý, trở thành một trong những hướng phát triển quan trọng của mạng nơ-ron đồ thị hiện đại.



Hình 6 : Mô hình GAT

2.2.2.1. Cơ chế Attention và hệ số chú ý

Trong Graph Attention Network (GAT), cơ chế *attention* đóng vai trò trung tâm trong việc xác định mức độ ảnh hưởng của từng đỉnh láng giềng khi tổng hợp thông tin cho một đỉnh cụ thể. Thay vì gán trọng số cố định như trong GCN, GAT học ra các hệ số chú ý thông qua một hàm học tham số, cho phép mô hình tự động đánh giá tầm quan trọng tương đối giữa các đỉnh liên kết. Cụ thể, với hai đỉnh liên kết i và j, GAT

đầu tiên chiếu đặc trưng của mỗi đỉnh thông qua một ma trận trọng số W để thu được biểu diễn mới. Sau đó, mức độ chú ý giữa hai đỉnh được tính bằng một hàm tương tác phi tuyến, thường là hàm kích hoạt *LeakyReLU*, ký hiệu là:

$$e_{ij} = LeakyReLU\left(a^T\left[Wh_j || Wh_j\right]\right)$$

trong đó a là vector trọng số của cơ chế chú ý, và || biểu diễn phép nối vector. Để đảm bảo tính chuẩn hóa và giúp việc so sánh giữa các đỉnh láng giềng trở nên hợp lý, GAT áp dụng hàm *softmax* để biến đổi các giá trị e_{ij} thành hệ số chú ý chuẩn hóa:

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k\epsilon\mathcal{N}(i)} \exp\,(e_{ik})}$$

Các hệ số chú ý α_{ij} này phản ánh mức độ quan trọng của đỉnh j đối với đỉnh i. Trong quá trình lan truyền thông tin, đặc trưng mới của đỉnh i được tính bằng tổng có trọng số của các đặc trưng đỉnh láng giềng, trong đó trọng số chính là các hệ số chú ý vừa được học.

Cơ chế *multi-head attention* thường được sử dụng để tăng tính ổn định và khả năng biểu diễn của mô hình. Thay vì chỉ có một tập hệ số chú ý, mô hình học đồng thời nhiều đầu chú ý khác nhau, mỗi đầu nắm bắt một khía cạnh riêng của mối quan hệ giữa các đỉnh. Các kết quả từ các đầu chú ý này sau đó được kết hợp để tạo thành biểu diễn cuối cùng của mỗi đỉnh. Cách tiếp cận này giúp GAT không chỉ học tốt hơn các cấu trúc đồ thị phức tạp mà còn giảm thiểu sự phụ thuộc vào một hướng chú ý duy nhất.

2.2.2.2. Ưu điểm và phạm vi ứng dụng của GAT

Một trong những ưu điểm nổi bật nhất của GAT là khả năng tự động học trọng số kết nối giữa các đỉnh trong đồ thị thông qua cơ chế *attention*. Trong khi các mô hình như GCN phải dựa vào ma trận kề cố định để xác định mức độ ảnh hưởng giữa các đỉnh, thì GAT cho phép mô hình tự điều chỉnh mức độ "chú ý" giữa các đỉnh lân cận dựa trên đặc trưng dữ liệu. Điều này giúp GAT xử lý hiệu quả hơn trong những trường hợp đồ thị không đồng nhất, có nhiều loại đỉnh hoặc cạnh khác nhau về ý nghĩa và tầm quan trọng.

Bên cạnh đó, cơ chế *multi-head attention* mang lại tính ổn định và khả năng biểu diễn mạnh mẽ cho GAT. Việc sử dụng nhiều "đầu chú ý" song song giúp mô hình học được các mối quan hệ khác nhau giữa các đỉnh, giảm thiểu rủi ro do một head bị học lệch. Ngoài ra, GAT còn có ưu thế trong việc mở rộng sang đồ thị động hoặc đồ thị lớn, bởi GAT không cần dựa vào toàn bộ ma trận kề, mà chỉ cần thông tin từ các đỉnh lân cận trong phạm vi tính toán attention. Điều này giúp giảm đáng kể độ phức tạp tính toán so với các mô hình GCN khi áp dụng trên các mạng lớn.

Về phạm vi ứng dụng, GAT được sử dụng rộng rãi trong nhiều lĩnh vực thực tế. Trong mạng xã hội, GAT giúp dự đoán mối quan hệ hoặc phát hiện cộng đồng bằng cách học trọng số giữa người dùng dựa trên mức độ tương tác. Trong xử lý ngôn ngữ tự nhiên, GAT có thể biểu diễn mối quan hệ ngữ nghĩa giữa các từ trong đồ thị phụ thuộc cú pháp. Ngoài ra, GAT cũng chứng minh hiệu quả trong phân loại nút, dự đoán liên kết và học biểu diễn đồ thị trong nhiều hệ thống khuyến nghị và phân tích dữ liệu lớn.

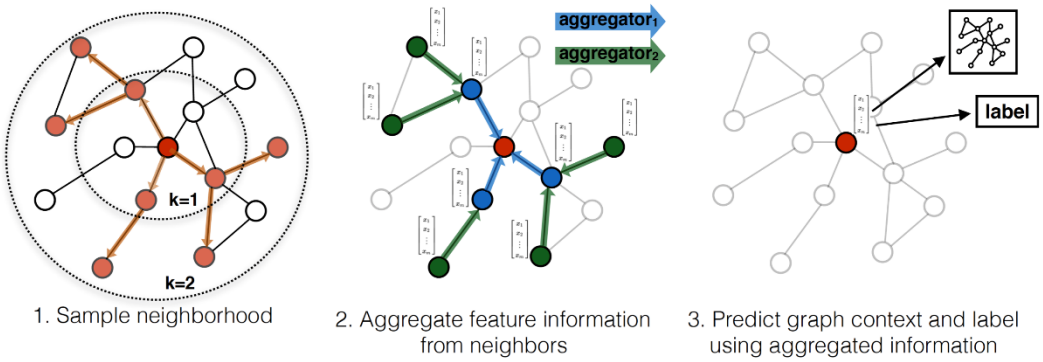
Tóm lại, GAT là một bước tiến quan trọng trong lĩnh vực mạng nơ-ron đồ thị. Nhờ khả năng học trọng số động và kết hợp nhiều hướng chú ý, GAT mang lại hiệu suất vượt trội trên nhiều loại đồ thị phức tạp, mở ra hướng nghiên cứu sâu hơn trong việc tối ưu hóa các mô hình GNN hướng chú ý và ứng dụng vào các bài toán thực tế.

2.2.3. GraphSAGE và các biến thể

GraphSAGE (Graph Sample and Aggregate) là một mô hình mạng nơ-ron đồ thị được đề xuất nhằm giải quyết vấn đề khả năng mở rộng khi áp dụng GNN trên các đồ thị có kích thước rất lớn. Thay vì yêu cầu toàn bộ đồ thị trong quá trình huấn luyện, GraphSAGE học biểu diễn nút thông qua việc lấy mẫu một tập con lân cận và tổng hợp thông tin từ chúng. Cách tiếp cận này giúp giảm đáng kể chi phí tính toán và bộ nhớ, cho phép mô hình huấn luyện trên các tập dữ liệu đồ thị quy mô công nghiệp.

Bên cạnh đó, GraphSAGE còn tạo tiền đề cho nhiều biến thể và cải tiến sau này, như việc áp dụng các chiến lược tổng hợp đặc trưng khác nhau hoặc cơ chế lấy mẫu thích ứng. Nhờ tính linh hoạt và hiệu quả trong xử lý, GraphSAGE đã trở thành một

trong những kiến trúc GNN được ứng dụng rộng rãi nhất trong các lĩnh vực như phân loại nút, dự đoán liên kết và khuyến nghị cá nhân hóa.



Hình 7 : Quy trình hoạt động của mô hình GraphSAGE

2.2.3.1. Kỹ thuật lấy mẫu trong đồ thị lớn

Một trong những thách thức chính của các mô hình GNN truyền thống là độ phức tạp tăng theo cấp số nhân khi số lượng tầng và số lượng láng giềng của mỗi nút tăng lên. Khi huấn luyện trên các đồ thị lớn như mạng xã hội hoặc đồ thị tri thức, việc lan truyền thông tin qua toàn bộ các nút lân cận sẽ nhanh chóng dẫn đến tình trạng bùng nổ bộ nhớ và chi phí tính toán cao. Để giải quyết vấn đề này, GraphSAGE giới thiệu kỹ thuật lấy mẫu láng giềng, cho phép mô hình chỉ chọn ngẫu nhiên một số lượng nút láng giềng cố định để lan truyền thông tin thay vì sử dụng toàn bộ.

Cụ thể, tại mỗi tầng của mạng, GraphSAGE không thu thập thông tin từ toàn bộ các nút kề mà lấy mẫu một tập con láng giềng. Quá trình này được thực hiện theo hướng từ dưới lên, tức là việc lấy mẫu được áp dụng lặp lại qua từng tầng, đảm bảo rằng phạm vi lan truyền thông tin được kiểm soát và không vượt quá giới hạn bộ nhớ. Việc lựa chọn láng giềng có thể được thực hiện theo nhiều chiến lược, chẳng hạn như lấy mẫu ngẫu nhiên đồng đều, lấy mẫu có trọng số dựa trên độ quan trọng, hoặc lấy mẫu thích ứng dựa trên đặc trưng học được trong quá trình huấn luyện.

Nhờ phương pháp này, GraphSAGE có khả năng huấn luyện trên đồ thị quy mô lớn mà không cần tải toàn bộ đồ thị vào bộ nhớ. Điều này giúp mô hình đạt được khả năng mở rộng cao, đồng thời vẫn duy trì được chất lượng biểu diễn của các nút. Tuy nhiên, việc lấy mẫu ngẫu nhiên cũng có thể dẫn đến sai lệch thống kê nếu số lượng mẫu quá nhỏ, nên việc cân bằng giữa hiệu suất và độ chính xác là yếu tố then chốt trong thiết kế mô hình GraphSAGE.

2.2.3.2. Các hàm kết hợp đặc trưng (Mean, LSTM, Pooling)

Trong kiến trúc của GraphSAGE, sau khi đã tiến hành lấy mẫu các nút láng giềng để giảm tải tính toán, bước tiếp theo quan trọng là tổng hợp thông tin từ các nút láng giềng nhằm tạo ra biểu diễn đặc trưng mới cho mỗi nút. Quá trình này giúp mô hình có thể nắm bắt được thông tin ngữ cảnh xung quanh từng nút mà không cần duyệt toàn bộ đồ thị. Cơ chế tổng hợp đặc trưng là trung tâm của GraphSAGE, bởi cơ chế đó quyết định cách thông tin được truyền và hòa trộn giữa các nút trong quá trình huấn luyện. GraphSAGE không giới hạn ở một kiểu tổng hợp duy nhất mà cung cấp nhiều hàm kết hợp đặc trưng khác nhau, giúp mô hình linh hoạt trong việc biểu diễn cấu trúc và mối quan hệ giữa các nút. Mỗi hàm có cách tiếp cận riêng để kết hợp thông tin từ các láng giềng, phù hợp với các loại dữ liệu và mục tiêu học khác nhau.

Mean Aggregator là phương pháp đơn giản và phổ biến nhất. Ở đây, biểu diễn của một nút được tính bằng cách lấy trung bình cộng của các vector đặc trưng từ các nút láng giềng, sau đó kết hợp với đặc trưng của chính nút đó. Cách tiếp cận này giúp duy trì tính ổn định và dễ huấn luyện, đồng thời giảm chi phí tính toán khi làm việc với đồ thị lớn. Mean aggregator có thể được xem như một biến thể mở rộng của GCN, nhưng có khả năng tổng quát hơn khi áp dụng cho đồ thị có cấu trúc động.

LSTM Aggregator sử dụng mạng nơ-ron hồi tiếp LSTM để xử lý thông tin từ các nút láng giềng theo một trình tự ngẫu nhiên. Dù đồ thị vốn không có thứ tự giữa các láng giềng, việc ngẫu nhiên hóa này vẫn cho phép LSTM học được các mẫu phụ thuộc phi tuyến và mối quan hệ phức tạp hơn. Phương pháp này đặc biệt hiệu quả khi dữ liệu đồ thị có nhiều mối quan hệ không tuyến tính hoặc cần học sâu về tương tác giữa các nút.

Pooling Aggregator hoạt động bằng cách truyền từng vector đặc trưng của nút láng giềng qua một mạng con phi tuyến, sau đó áp dụng một hàm gộp như max-pooling hoặc mean-pooling để trích xuất thông tin nổi bật nhất. Cách tiếp cận này cho phép mô hình học được các đặc trưng quan trọng nhất của tập láng giềng, nhấn mạnh vào các nút có vai trò nổi bật trong việc xác định biểu diễn của nút trung tâm.

Nhìn chung, các hàm kết hợp đặc trưng giúp GraphSAGE linh hoạt trong việc biểu diễn thông tin từ đồ thị, cân bằng giữa độ chính xác và chi phí tính toán. Trong thực tế,

Mean aggregator thường được lựa chọn cho các bài toán lớn và cần tốc độ, trong khi LSTM và Pooling aggregator phù hợp với các bài toán phức tạp đòi hỏi khả năng biểu diễn cao hơn.

2.3. Kết luận

Chương này đã trình bày cơ sở lý thuyết và các mô hình tiêu biểu của mạng nơ-ron đồ thị, một hướng tiếp cận quan trọng trong lĩnh vực học sâu trên dữ liệu phi cấu trúc. Ở phần đầu, nội dung tập trung vào việc mô tả biểu diễn toán học của đồ thị và cơ chế lan truyền thông tin - nền tảng giúp GNN có khả năng học được mối quan hệ phụ thuộc giữa các nút và cạnh trong không gian phi Euclid. Tiếp đó, kiến trúc tổng quát của GNN được trình bày như một quy trình thống nhất, bao gồm các bước thu thập thông tin từ láng giềng, cập nhật biểu diễn nút và huấn luyện mô hình toàn cục.

Phần tiếp theo đã đi sâu vào các mô hình GNN phổ biến, bao gồm GCN, GAT và GraphSAGE. Mỗi mô hình thể hiện một phương pháp khác nhau trong việc xử lý và tổng hợp thông tin trên đồ thị. GCN kế thừa nguyên lý tích chập từ mạng CNN và áp dụng trên cấu trúc đồ thị, cho phép học biểu diễn đặc trưng của nút thông qua trung bình có trọng số của các nút láng giềng. GAT giới thiệu cơ chế chú ý nhằm xác định mức độ quan trọng tương đối của từng nút láng giềng, giúp cải thiện khả năng biểu diễn trong các đồ thị không đồng nhất. Trong khi đó, GraphSAGE hướng đến khả năng mở rộng bằng cách lấy mẫu và tổng hợp đặc trưng, phù hợp với các đồ thị quy mô lớn và có tính động cao.

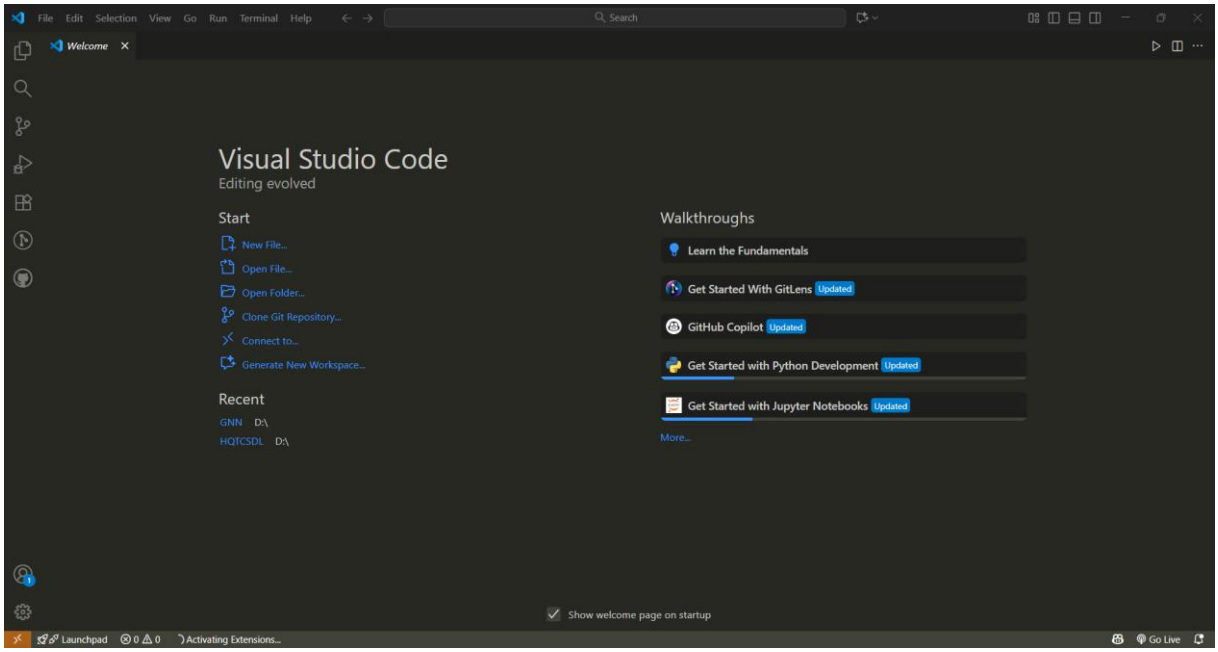
Qua phân tích, có thể nhận thấy rằng mỗi mô hình GNN đều có ưu điểm và giới hạn riêng, phụ thuộc vào đặc trưng của dữ liệu và mục tiêu ứng dụng. Tuy nhiên, điểm chung nổi bật của các mô hình này là khả năng khai thác mối quan hệ cấu trúc phức tạp giữa các thực thể, cho phép cải thiện hiệu quả học biểu diễn và phân loại trong nhiều lĩnh vực khác nhau.

CHƯƠNG 3: MÔ PHỎNG VÀ THỬ NGHIỆM CÔNG NGHỆ MẠNG NƠ-RON ĐỒ THỊ

3.1. Giới thiệu phần mềm và công cụ hỗ trợ quá trình mô phỏng

3.1.1. Visual Studio Code

Visual Studio Code là môi trường phát triển tích hợp được sử dụng chính trong quá trình viết và thực thi chương trình mô phỏng mạng nơ-ron đồ thị (GNN). VS Code do Microsoft phát triển, là một công cụ mã nguồn mở, miễn phí, hỗ trợ đa nền tảng (Windows, macOS, Linux) và được đánh giá cao bởi tính linh hoạt, tốc độ xử lý nhanh cùng hệ sinh thái mở rộng phong phú.



Hình 8 : Visual Studio Code

Trong nghiên cứu này, VS Code được sử dụng để phát triển và chạy file mô phỏng GCN.py. Giao diện thân thiện và hỗ trợ các tính năng như tự động hoàn thiện mã lệnh, kiểm tra lỗi cú pháp và gỡ lỗi giúp quá trình lập trình và kiểm thử trở nên thuận tiện hơn. Bên cạnh đó, VS Code tích hợp sẵn terminal nội bộ, cho phép người dùng chạy trực tiếp các lệnh Python, quản lý môi trường ảo, cài đặt thư viện thông qua pip, cũng như theo dõi log huấn luyện của mô hình mà không cần chuyển đổi sang công cụ khác.

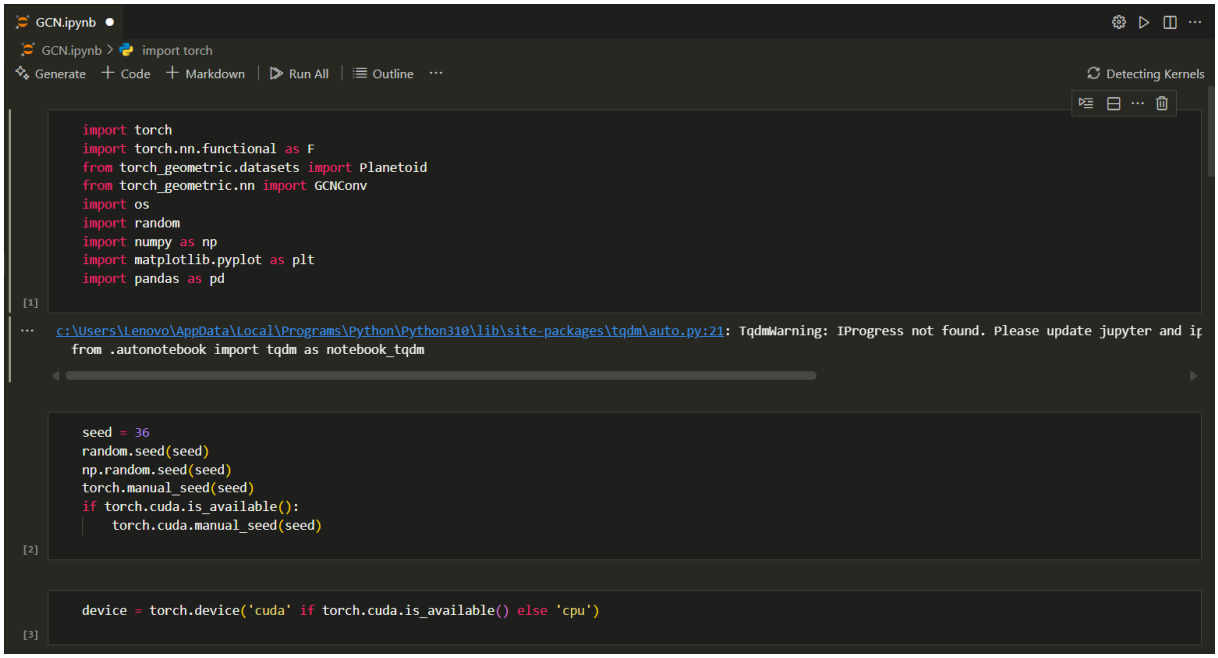
Một ưu điểm quan trọng khác của VS Code là khả năng tích hợp mạnh mẽ với các extension chuyên biệt cho Python và PyTorch, như Python Extension Pack, Pylance, và

Jupyter. Nhờ đó, người dùng có thể kết hợp linh hoạt giữa việc viết mã dạng script và thử nghiệm từng khối mã nhỏ dạng notebook trong cùng một môi trường. Điều này đặc biệt hữu ích trong mô phỏng GNN, nơi cần liên tục quan sát giá trị loss, accuracy hoặc cấu trúc mạng trong quá trình huấn luyện.

Nhờ những ưu điểm về hiệu năng, tính mở rộng và khả năng tích hợp sâu với hệ sinh thái Python, Visual Studio Code là lựa chọn tối ưu cho việc phát triển và triển khai mô phỏng mạng nơ-ron đồ thị trong nghiên cứu này.

3.1.2. Jupyter Notebook

Jupyter Notebook là công cụ được sử dụng song song với Visual Studio Code trong quá trình mô phỏng và thử nghiệm mô hình mạng nơ-ron đồ thị (GNN). Đây là một môi trường tương tác mạnh mẽ cho phép kết hợp giữa mã nguồn, kết quả thực thi và nội dung mô tả trong cùng một tài liệu. Jupyter đặc biệt phù hợp cho các dự án nghiên cứu học máy và khoa học dữ liệu, nơi người dùng cần thực hiện các bước thử nghiệm, quan sát kết quả và điều chỉnh tham số một cách linh hoạt.



Hình 9 : Jupyter Notebook

Trong nghiên cứu này, Jupyter Notebook được dùng để triển khai và kiểm thử mã nguồn của mô hình Graph Convolutional Network (GCN). Thay vì chạy toàn bộ chương trình cùng lúc, công cụ này cho phép thực thi từng khối mã độc lập, giúp người nghiên cứu dễ dàng theo dõi biến trung gian, biểu đồ tổn thất, độ chính xác hoặc kiểm tra cấu

trúc của đồ thị tại các giai đoạn khác nhau. Cách tiếp cận này rất hữu ích trong quá trình tinh chỉnh siêu tham số và đánh giá tác động của các thay đổi nhỏ lên kết quả huấn luyện.

Một ưu điểm nổi bật của Jupyter Notebook là khả năng trực quan hóa dữ liệu ngay trong môi trường làm việc. Với sự kết hợp của các thư viện như Matplotlib hoặc NetworkX, người dùng có thể vẽ biểu đồ thể hiện mối quan hệ giữa các nút trong đồ thị, hiển thị đường cong Loss - Epoch và Accuracy - Epoch, hoặc so sánh kết quả giữa các lần huấn luyện khác nhau. Điều này giúp minh họa rõ ràng quá trình lan truyền thông tin và hội tụ của mô hình GCN, đồng thời hỗ trợ phân tích kết quả mô phỏng một cách trực quan, thuyết phục.

Nhờ tính linh hoạt, khả năng trực quan hóa mạnh và sự tương thích cao với các thư viện học sâu, Jupyter Notebook trở thành công cụ không thể thiếu trong quá trình phát triển, kiểm thử và minh họa mô hình GNN trong nghiên cứu này.

3.1.3. Thư viện hỗ trợ

Trong quá trình mô phỏng và huấn luyện mô hình mạng nơ-ron đồ thị (GNN), hệ thống thư viện Python đóng vai trò nền tảng, cung cấp các công cụ giúp xử lý dữ liệu, xây dựng mô hình, tối ưu tham số và trực quan hóa kết quả. Những thư viện này không chỉ giúp giảm thiểu khối lượng lập trình thủ công mà còn đảm bảo tính chính xác và hiệu quả trong tính toán. Dưới đây là các nhóm thư viện chính được sử dụng trong quá trình nghiên cứu và mô phỏng.

Thứ nhất, thư viện cốt lõi được sử dụng là PyTorch và PyTorch Geometric (PyG). PyTorch là nền tảng học sâu mã nguồn mở được phát triển bởi Facebook AI Research, cho phép định nghĩa các mạng nơ-ron một cách linh hoạt. Dựa trên nền tảng đó, PyTorch Geometric mở rộng khả năng xử lý dữ liệu sang miền đồ thị, cung cấp nhiều lớp và hàm chuyên dụng phục vụ việc xây dựng các kiến trúc mạng nơ-ron đồ thị khác nhau. Thư viện này còn hỗ trợ sẵn nhiều bộ dữ liệu phổ biến như Cora, Citeseer, … giúp quá trình thử nghiệm và đánh giá mô hình diễn ra nhanh chóng, nhất quán và dễ dàng tái hiện.

Thứ hai, nhóm thư viện xử lý và thao tác dữ liệu gồm NumPy và Pandas. NumPy hỗ trợ tính toán ma trận, vector và các phép toán tuyến tính với hiệu năng cao, đặc biệt cần thiết trong việc biểu diễn ma trận kề và ma trận đặc trưng của đồ thị. Trong khi đó,

Pandas giúp quản lý, lọc và phân tích dữ liệu đầu vào ở dạng bảng, đảm bảo các bước tiền xử lý như chuẩn hóa, tách dữ liệu huấn luyện - kiểm thử được thực hiện hiệu quả và có hệ thống.

Thứ ba, nhóm thư viện trực quan hóa đóng vai trò quan trọng trong việc trình bày kết quả mô phỏng. Matplotlib được sử dụng để vẽ biểu đồ thể hiện mối quan hệ giữa *Loss - Epoch* và *Accuracy - Epoch*, giúp đánh giá tốc độ hội tụ và hiệu năng của mô hình trong quá trình huấn luyện. Bên cạnh đó, NetworkX hỗ trợ hiển thị cấu trúc đồ thị, minh họa mối quan hệ giữa các nút và cạnh, giúp người quan sát dễ dàng hình dung quá trình lan truyền thông tin trong mạng.

Cuối cùng, một số công cụ bổ trợ như Scikit-learn được sử dụng trong các bước đánh giá mô hình, tính toán các chỉ số độ chính xác, độ nhạy, và ma trận nhầm lẫn. Các thư viện này giúp quá trình đánh giá được thực hiện khách quan và bám sát chuẩn mực trong lĩnh vực học máy.

Nhờ sự kết hợp hài hòa giữa PyTorch Geometric - nền tảng học sâu đồ thị mạnh mẽ, NumPy/Pandas - công cụ xử lý dữ liệu hiệu quả, và Matplotlib/NetworkX - các công cụ trực quan hóa trực tiếp, quá trình mô phỏng mô hình GCN trong nghiên cứu đạt được sự cân bằng giữa độ chính xác, tính linh hoạt và khả năng minh họa trực quan, đảm bảo tính khoa học và khả năng mở rộng cho các nghiên cứu tiếp theo.

3.2. Cài đặt và thiết lập mô phỏng

3.2.1. Chuẩn bị dữ liệu

Bộ dữ liệu được sử dụng trong quá trình mô phỏng là Cora dataset, một trong những tập dữ liệu kinh điển và phổ biến nhất trong lĩnh vực nghiên cứu về Graph Neural Network (GNN). Cora là một đồ thị trích dẫn học thuật, trong đó mỗi nút biểu diễn một bài báo khoa học, còn mỗi cạnh đại diện cho mối quan hệ trích dẫn giữa hai bài báo. Mục tiêu của mô hình là phân loại các nút theo chủ đề học thuật mà bài báo thuộc về, dựa trên thông tin về nội dung và mối quan hệ trích dẫn giữa chúng.

Cụ thể, tập dữ liệu Cora bao gồm 2.708 nút, tương ứng với 2.708 bài báo, được chia thành 7 lớp chủ đề như Machine Learning, Information Retrieval, Neural Networks, v.v. Mỗi nút được mô tả bởi một vector đặc trưng gồm 1.433 thuộc tính, phản ánh tần

suất xuất hiện của các từ khóa trong nội dung bài báo. Ngoài ra, Cora có tổng cộng 5.429 cạnh, thể hiện mối quan hệ "bài báo A trích dẫn bài báo B". Cấu trúc liên kết này giúp GCN khai thác được mối quan hệ ngữ nghĩa và cấu trúc mạng trong quá trình học.

```
data_root = os.path.join(os.getcwd(), 'data')
dataset = Planetoid(root=data_root, name='Cora')   # Cora: node-classification benchmark
data = dataset[0].to(device)
```

Hình 10 : Lấy dataset về để mô phỏng

Trước khi đưa vào huấn luyện, dữ liệu được chuẩn hóa đặc trưng để đảm bảo các giá trị trong vector đầu vào có cùng tỷ lệ, tránh việc một thuộc tính có độ lớn vượt trội gây ảnh hưởng đến kết quả học. PyTorch Geometric hỗ trợ các phương thức xử lý này tự động, giúp đảm bảo tính nhất quán trong toàn bộ quá trình huấn luyện.

Như vậy, giai đoạn chuẩn bị dữ liệu đóng vai trò quan trọng trong toàn bộ quy trình mô phỏng. Việc lựa chọn Cora dataset không chỉ giúp mô hình có dữ liệu đủ phức tạp để đánh giá năng lực học biểu diễn của GCN mà còn đảm bảo khả năng so sánh kết quả với các công trình nghiên cứu trước đây - bởi Cora là bộ dữ liệu tham chiếu tiêu chuẩn trong lĩnh vực học sâu trên đồ thị.

3.2.2. Huấn luyện mô hình và kết quả

Sau khi hoàn tất quá trình chuẩn bị dữ liệu, bước tiếp theo trong mô phỏng là xây dựng và huấn luyện mô hình Graph Convolutional Network (GCN). Mục tiêu của giai đoạn này là giúp mô hình học được biểu diễn đặc trưng của các nút trong đồ thị sao cho các nút có mối quan hệ liên kết hoặc thuộc cùng một chủ đề học thuật sẽ có đặc trưng tương tự nhau trong không gian biểu diễn.

Trong mã nguồn mô phỏng, mô hình được xây dựng thông qua lớp Net. Mô hình gồm hai tầng tích chập đồ thị kết hợp với hàm kích hoạt phi tuyến ReLU và kỹ thuật loại bỏ ngẫu nhiên nơ-ron nhằm tránh hiện tượng quá khớp. Tầng thứ nhất có nhiệm vụ trích xuất đặc trưng ẩn từ ma trận đầu vào, trong khi tầng thứ hai thực hiện ánh xạ đặc trưng sang không gian nhãn đầu ra, tương ứng với số lớp phân loại trong bộ dữ liệu Cora. Đầu ra cuối cùng được đưa qua hàm log_softmax để chuyển đổi thành xác suất log của từng lớp. Cấu trúc mô hình trong mã GCN được định nghĩa như sau:

```python
class GCN(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels, dropout=0.5):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)
        self.dropout = dropout

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)
        x = self.conv2(x, edge_index)
        return x
```

Hình 11 : Định nghĩa lớp mô hình GCN

Sau khi định nghĩa kiến trúc mô hình, quá trình huấn luyện được thiết lập bằng cách chọn thiết bị tính toán, khởi tạo mô hình và dữ liệu, đồng thời cấu hình hàm tối ưu Adam với tốc độ học 0.01 và hệ số điều chuẩn weight_decay để kiểm soát độ lớn của trọng số. Đoạn mã sau được dùng để khởi tạo và chuẩn bị mô hình huấn luyện:

```python
optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)
```

Hình 12 : Cấu hình tối ưu trong quá trình huấn luyện

Quá trình huấn luyện mô hình được định nghĩa trong hàm train(), trong đó mỗi lần lặp mô hình thực hiện lan truyền thuận, tính toán hàm mất mát theo Negative Log Likelihood (NLLLoss), sau đó lan truyền ngược để cập nhật trọng số bằng thuật toán Adam. Quá trình này được lặp lại giúp mô hình dần hội tụ và tối ưu hóa khả năng dự đoán.

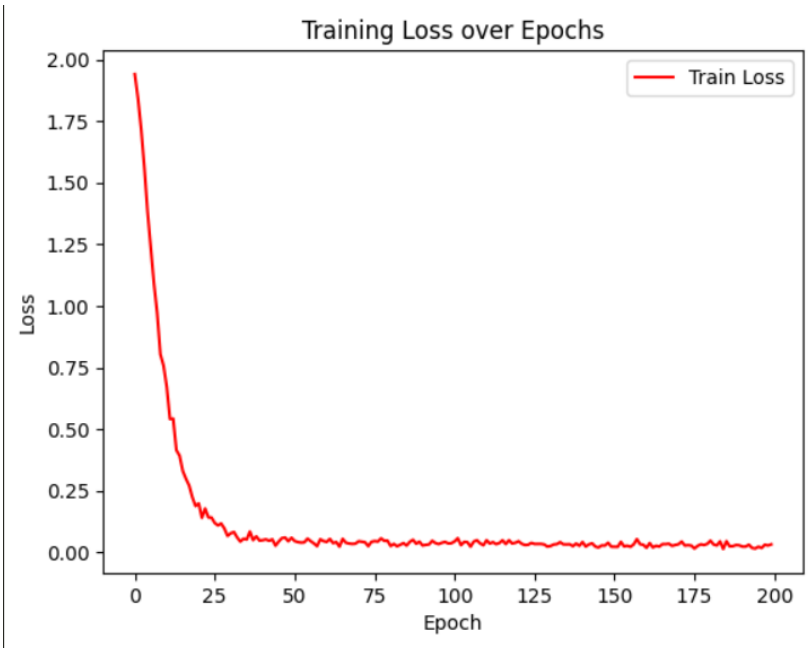
```python
def train():
    model.train()
    optimizer.zero_grad()
    out = model(data)
    loss = F.nll_loss(out[data.train_mask], data.y[data.train_mask])
    loss.backward()
    optimizer.step()
    return loss

for epoch in range(1, 201):
    loss = train()
    print(f'Epoch: {epoch:03d}, Loss: {loss:.4f}')
```

Hình 13 : Quy trình huấn luyện mô hình GCN qua các epoch

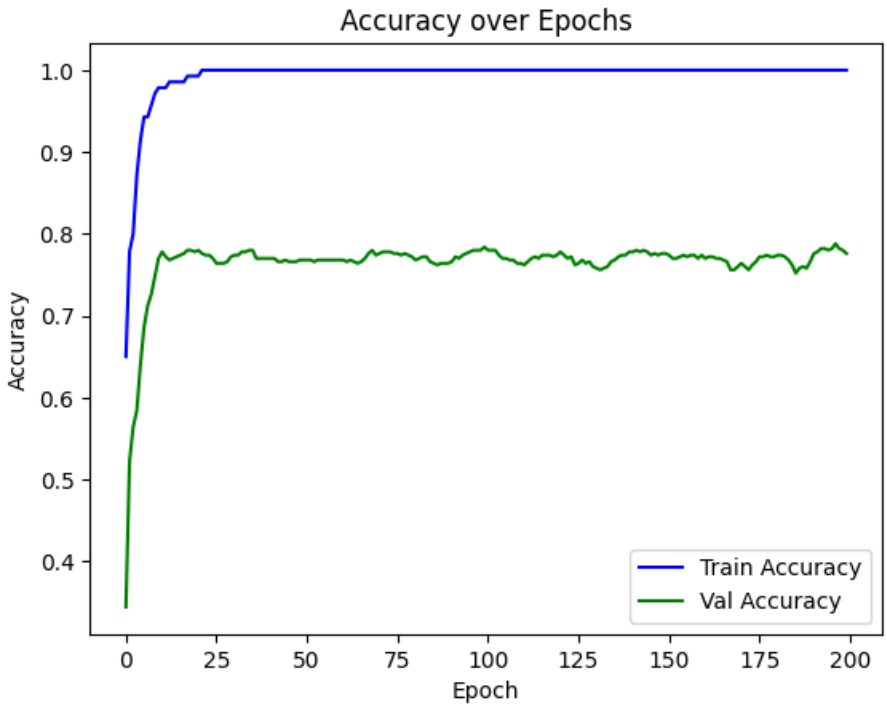
Trong mỗi vòng lặp, hàm train() đưa mô hình về chế độ huấn luyện, xóa gradient cũ, tính đầu ra out thông qua hàm forward(), sau đó tính giá trị mất mát dựa trên tập huấn luyện (train_mask). Tiếp theo, hàm loss.backward() lan truyền gradient ngược qua các tầng mạng, và optimizer.step() cập nhật lại trọng số mô hình. Giá trị mất mát được in ra sau mỗi vòng giúp theo dõi quá trình hội tụ trong suốt quá trình huấn luyện.

Trong giai đoạn đầu, giá trị loss giảm rất nhanh từ khoảng 2.0 xuống dưới 0.3 chỉ sau 20 epoch, sau đó tiếp tục giảm dần và ổn định gần mức 0 khi kết thúc huấn luyện. Biểu đồ "Training Loss over Epochs" cho thấy đường cong loss có xu hướng giảm mượt,



Hình 14 : Biểu đồ giá trị hàm mất mát

không dao động mạnh, chứng tỏ quá trình hội tụ ổn định và mô hình không gặp khó khăn trong việc tối ưu hóa trọng số.



Hình 15 : Biểu đồ độ chính xác của mô hình

Kết quả huấn luyện cũng được đánh giá thông qua độ chính xác (accuracy) của mô hình trên cả tập huấn luyện và tập kiểm định. Hai đường biểu diễn trong biểu đồ "Accuracy over Epochs" cho thấy độ chính xác trên tập huấn luyện tăng nhanh và đạt giá trị xấp xỉ tối đa chỉ sau khoảng 30 epoch, trong khi độ chính xác trên tập kiểm định đạt mức ổn định quanh 0.78 - 0.80 trong phần lớn thời gian huấn luyện.

Hiện tượng này cho thấy mô hình đã học tốt cấu trúc của dữ liệu huấn luyện, song có dấu hiệu overfitting - mô hình ghi nhớ dữ liệu huấn luyện rất tốt nhưng khả năng tổng quát hóa sang dữ liệu mới còn hạn chế. Điều này là phổ biến trong các thí nghiệm sử dụng bộ dữ liệu Cora, vốn có số lượng nút và cạnh tương đối nhỏ. Tuy nhiên, kết quả vẫn thể hiện mô hình hội tụ tốt và hoạt động ổn định, thể hiện khả năng học đặc trưng hiệu quả của GCN.

Quan sát từ hai biểu đồ cho thấy mối quan hệ nghịch lý tưởng giữa Loss và Accuracy: khi loss giảm dần, độ chính xác tăng lên và duy trì ổn định, phản ánh một quá trình học hiệu quả. Không xuất hiện tình trạng dao động mạnh hay suy giảm đột ngột ở

giai đoạn sau huấn luyện, chứng minh rằng việc sử dụng ReLU, Dropout, và tối ưu hóa bằng Adam đã mang lại sự ổn định cho quá trình lan truyền và cập nhật trọng số.

Tổng thể, kết quả huấn luyện và đánh giá cho thấy mô hình GCN trong mô phỏng này đã học được mối quan hệ giữa các nút và cấu trúc liên kết trong đồ thị Cora một cách chính xác, thể hiện khả năng trích xuất đặc trưng mạnh mẽ. Mặc dù còn xuất hiện dấu hiệu quá khớp nhẹ, nhưng quá trình huấn luyện đạt được sự hội tụ nhanh, ổn định và đáng tin cậy, chứng minh tính đúng đắn và hiệu quả của mô hình GCN trong xử lý dữ liệu đồ thị.

3.3. Kết luận

Việc mô phỏng mạng nơ-ron đồ thị (GNN) bằng mô hình GCN trên môi trường thực nghiệm đã thể hiện rõ tiềm năng của phương pháp học sâu trong việc xử lý dữ liệu có cấu trúc đồ thị. Thông qua quá trình huấn luyện và đánh giá mô hình trên tập dữ liệu thực nghiệm, kết quả thu được cho thấy GCN có khả năng khai thác hiệu quả mối quan hệ giữa các nút và cạnh, từ đó học được biểu diễn đặc trưng mang tính toàn cục và cục bộ của đồ thị.

Các bước thiết lập, huấn luyện và trực quan hóa kết quả đã phản ánh được cơ chế lan truyền thông tin và quá trình hội tụ của mô hình. Việc quan sát sự thay đổi của độ tổn thất và độ chính xác theo từng epoch giúp xác định mức độ ổn định cũng như khả năng khái quát hóa của GCN. Những kết quả này khẳng định vai trò quan trọng của việc lựa chọn tham số huấn luyện, cấu trúc mô hình, cũng như chiến lược tối ưu trong việc nâng cao hiệu quả học của mạng nơ-ron đồ thị.

Từ các kết quả mô phỏng, có thể nhận thấy rằng GCN là nền tảng vững chắc cho nhiều hướng mở rộng khác như GAT hay GraphSAGE. Các nghiên cứu tương lai có thể tập trung vào việc mở rộng khả năng xử lý trên đồ thị quy mô lớn, tăng tính linh hoạt trong lan truyền thông tin, hoặc kết hợp cơ chế chú ý để cải thiện hiệu năng mô hình trong các bài toán thực tiễn như phân tích sinh học, khai phá mạng xã hội và nhận dạng mẫu phức tạp.

TỔNG KẾT

Đề tài đã tiến hành một nghiên cứu toàn diện nhằm tìm hiểu bản chất, cơ chế hoạt động và tiềm năng ứng dụng của mạng nơ-ron đồ thị trong bối cảnh phát triển mạnh mẽ của trí tuệ nhân tạo hiện nay. Trong khi các mô hình học sâu truyền thống thường giới hạn trong dữ liệu có cấu trúc Euclid, GNN đánh dấu một bước tiến quan trọng khi có khả năng xử lý dữ liệu phi cấu trúc dưới dạng đồ thị - nơi thông tin được thể hiện không chỉ qua từng thực thể mà còn thông qua mối liên kết giữa chúng. Nghiên cứu đã hệ thống hóa các khái niệm nền tảng như nút, cạnh, ma trận kề và cơ chế lan truyền thông tin, qua đó làm rõ cơ sở lý thuyết giúp GNN mô hình hóa các mối quan hệ phức tạp và học biểu diễn đặc trưng từ cấu trúc đồ thị.

Trên nền tảng lý thuyết đó, đề tài đã phân tích ba mô hình tiêu biểu gồm GCN, GAT và GraphSAGE, thể hiện rõ sự tiến hóa của GNN từ những phương pháp tích chập truyền thống đến các kiến trúc linh hoạt hơn dựa trên cơ chế chú ý và kỹ thuật lấy mẫu. Các mô hình được mô tả và so sánh toàn diện, tập trung vào cơ chế lan truyền thông tin, khả năng học biểu diễn đặc trưng và giới hạn về hiệu năng tính toán. Phần mô phỏng thực nghiệm, được triển khai đã minh họa cụ thể quy trình huấn luyện mô hình GCN, bao gồm tiền xử lý dữ liệu, thiết lập tham số và đánh giá kết quả thông qua đồ thị biểu diễn độ chính xác và hàm mất mát. Kết quả trực quan từ mô phỏng cho thấy GCN có khả năng khai thác hiệu quả mối quan hệ giữa các nút và cạnh, giúp mô hình học được đặc trưng mang tính cấu trúc của dữ liệu, từ đó chứng minh tính khả thi và tiềm năng ứng dụng của GNN trong các bài toán phân loại và nhận dạng mẫu phức tạp.

Tổng thể, đề tài đã góp phần củng cố nền tảng lý thuyết về mạng nơ-ron đồ thị, đồng thời khẳng định vai trò của GNN như một hướng phát triển chiến lược trong học sâu hiện đại. Công nghệ này mở ra khả năng ứng dụng rộng rãi trong nhiều lĩnh vực như sinh học tính toán, mạng xã hội, tài chính, phân tích mạng giao thông và hệ thống khuyến nghị, nơi các mối quan hệ đóng vai trò trung tâm trong việc suy luận và ra quyết định. Trong tương lai, việc kết hợp GNN với các mô hình tiên tiến như Graph Transformer, Dynamic GNN hay các cơ chế học kết hợp khuếch tán sẽ mở rộng hơn nữa khả năng biểu diễn và học của mạng, hướng tới việc xây dựng những hệ thống trí tuệ nhân tạo có khả năng hiểu và xử lý thông tin phức tạp như con người.

TÀI LIỆU THAM KHẢO

[1] Jiaqi HAN, Jiacheng CEN, Liming WU, Zongzhao LI, Xiangzhe KONG, Rui JIAO, Ziyang YU, Tingyang XU, Fandi WU, Zihe WANG, Hongteng XU, Zhewei WEI, Deli ZHAO, Yang LIU, Yu RONG, Wenbing HUANG, "A survey of geometric graph neural networks: data structures, models and applications," 28 May, 2025.

[2] Haoran Lu, Lei Wang, Xiaoliang Ma, Jun Cheng, Mengchu Zhou, "A survey of graph neural networks and their industrial applications," 2024.

[3] M. W. Thomas N. Kipf, "SEMI-SUPERVISED CLASSIFICATION WITH GRAPH CONVOLUTIONAL NETWORKS," 2017.

[4] R. Y. J. L. William L. Hamilton, "Inductive Representation Learning on Large Graphs," 2017.

[5] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio`, Yoshua Bengio, "GRAPH ATTENTION NETWORKS," 2018.

[6] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, Maosong Sun, "Graph neural networks: A review of methods and applications," 2020.

[7] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, Philip S. Yu, "A Comprehensive Survey on Graph Neural Networks," 2019.